AI Applications Lecture 12

Image Generation 2: VAE as an interface to the real data space

SUZUKI, Atsushi[a]

[a] I thank my collaborator, Jing WANG, for cooperating in creating the teaching materials.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. When does the latent space strategy work?

4. How do we adapt various resolutions? - fully convolutional VAE

5. How does a VAE learn? How does it connect the latent space and raw data space?

6. Summary and Next Lecture

Recap: pipeline components

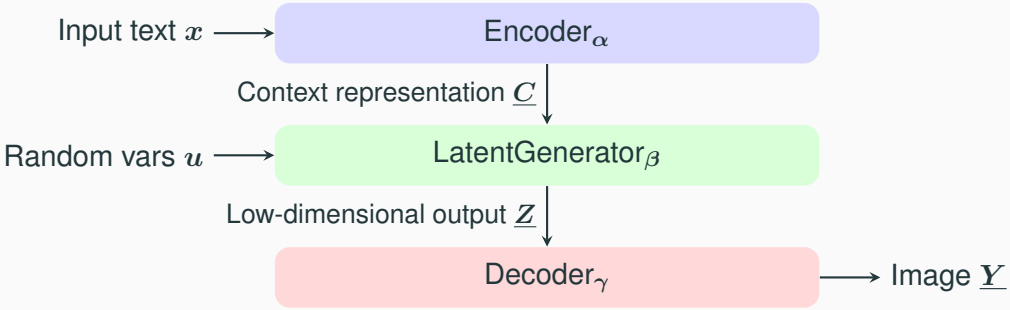


Figure 1: Latent generation pipeline.

Background

In recent image generation and 3D data generation, much attention tends to be paid to the iterative process of generating data from noise. However, a historically important turning point in the widespread adoption of such iterative generation pipelines was the practical realization of methods that perform the iterative generation process only in a low-dimensional latent space, rather than in the raw data space.

Background

In recent image generation and 3D data generation, much attention tends to be paid to the iterative process of generating data from noise. However, a historically important turning point in the widespread adoption of such iterative generation pipelines was the practical realization of methods that perform the iterative generation process only in a low-dimensional latent space, rather than in the raw data space.

A representative component that connects this low-dimensional latent space to the raw data space is the variational autoencoder (VAE).

Background

In recent image generation and 3D data generation, much attention tends to be paid to the iterative process of generating data from noise. However, a historically important turning point in the widespread adoption of such iterative generation pipelines was the practical realization of methods that perform the iterative generation process only in a low-dimensional latent space, rather than in the raw data space.

A representative component that connects this low-dimensional latent space to the raw data space is the variational autoencoder (VAE).

Reverse diffusion processes and flow matching are commonly used for generation in latent space, but in both cases, combining them with a VAE is the predominant approach. From this perspective, perhaps the VAE, rather than these generative processes, is the true essence of visual data generation.

Learning outcomes

At the end of this lecture, you will be able to

Learning outcomes

At the end of this lecture, you will be able to

- Decide whether to adopt the latent space strategy by e.g., variational autoencoder (VAE), depending on the data type in a new application.

Learning outcomes

At the end of this lecture, you will be able to

- Decide whether to adopt the latent space strategy by e.g., variational autoencoder (VAE), depending on the data type in a new application.
- Make your NN adaptable to variable output shape request by the user using **fully-convolutional** VAE.

Learning outcomes

At the end of this lecture, you will be able to

- Decide whether to adopt the latent space strategy by e.g., variational autoencoder (VAE), depending on the data type in a new application.
- Make your NN adaptable to variable output shape request by the user using **fully-convolutional** VAE.
- Explain the training phase of VAE and the difference between the original VAE and VAEs in visual data generation pipeline.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. When does the latent space strategy work?

4. How do we adapt various resolutions? - fully convolutional VAE

5. How does a VAE learn? How does it connect the latent space and raw data space?

6. Summary and Next Lecture

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. When does the latent space strategy work?

4. How do we adapt various resolutions? - fully convolutional VAE

5. How does a VAE learn? How does it connect the latent space and raw data space?

6. Summary and Next Lecture

Preliminaries: mathematical notations

Set:

- Sets: $\mathcal{A}$
- Membership: $x \in \mathcal{A}$
- Empty set: $\{\}$
- Roster notation: $\{a, b, c\}$
- Set-builder: $\{x \in \mathcal{A} | P(x)\}$
- Cardinality: $|\mathcal{A}|$
- Real numbers: $\mathbb{R}, \mathbb{R}_{>0}, \mathbb{R}_{\geq 0}$
- Integers: $\mathbb{Z}, \mathbb{Z}_{>0}, \mathbb{Z}_{\geq 0}$
- Integer range: $[1, k]_{\mathbb{Z}} \coloneqq \{1, \ldots, k\}$

Function:

- $f : \mathcal{X} \to \mathcal{Y}$: f maps from $\mathcal{X}$ to $\mathcal{Y}$.
- $y = f(x)$: The output of f for input x.

Definition:

- $(\mathrm{LHS}) \coloneqq (\mathrm{RHS})$: Left side is defined by the right side.

Preliminaries: mathematical notations

Sequence: Denoted by $\boldsymbol{a} = (a_1, a_2, \dots)$.

- A function $\boldsymbol{a} : [1, n]_{\mathbb{Z}} \to \mathcal{A}$.
- Length is denoted by $|\boldsymbol{a}|$.

Vector: Denoted by $\boldsymbol{v}$.

- A column of numbers, $\boldsymbol{v} \in \mathbb{R}^n$.
- i-th element is v_i.

Matrix: Denoted by $\boldsymbol{A}$.

- $m \times n$ matrix: $\boldsymbol{A} \in \mathbb{R}^{m,n}$.
- (i, j)-th element is $a_{i,j}$.
- Transpose: $\boldsymbol{A}^\top$.

Tensor: Denoted by $\underline{\boldsymbol{A}}$.

- Simply a multi-dimensional array.
- Vector $\to$ 1st-order, Matrix $\to$ 2nd-order.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. When does the latent space strategy work?

4. How do we adapt various resolutions? - fully convolutional VAE

5. How does a VAE learn? How does it connect the latent space and raw data space?

6. Summary and Next Lecture

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. When does the latent space strategy work?

4. How do we adapt various resolutions? - fully convolutional VAE

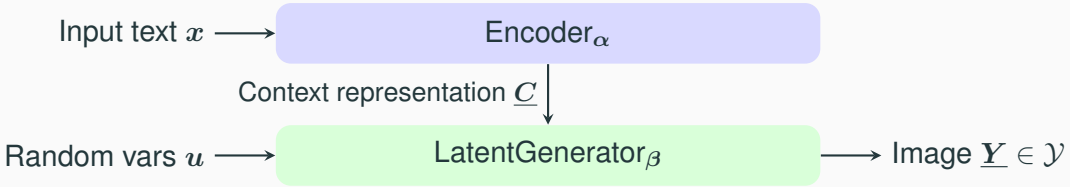
5. How does a VAE learn? How does it connect the latent space and raw data space?

6. Summary and Next Lecture

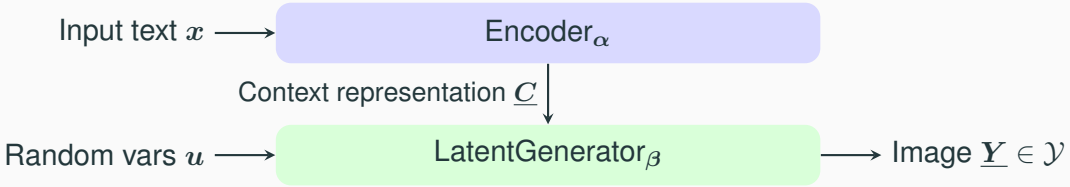
Question: If we use a latent space, instead of the raw data space, in the generation pipeline, we must sacrifice something. When

Question: If we use a latent space, instead of the raw data space, in the generation pipeline, we must sacrifice something. When does the latent-space-based pipeline work?

Motivation of using latent space and a VAE decoder

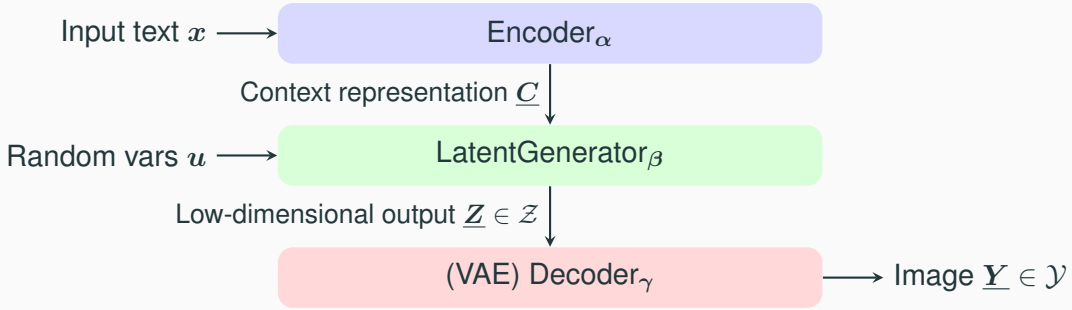


Motivation of using latent space and a VAE decoder



Without a (VAE) Decoder$_{\gamma}$, the other components need to work on the high-dimensional raw data space $\mathcal{Y}$.

Motivation of using latent space and a VAE decoder



Without a (VAE) Decoder$_\gamma$, the other components can work on the low-dimensional $\mathcal{Z}$.

Computational cost reduction effect of VAE Decoder

Raw image pixel space:

- Height: H
- Width: W
- Channel: 3

Latent low-dimensional space:

- Height: $\frac{H}{s}$
- Width: $\frac{W}{s}$
- Channel: C_{latent}

In a typical Stable Diffusion v1 series, downsampling factor is $s = 8$ and the number of channels is $C_{\text{latent}} = 4$.

The (time and space) **computational cost** of a latent-space UNet is approximately proportional to $\frac{1}{s^2}$.

If we focus on the effect of $s = 8$, in the Stable Diffusion v1, the latent space strategy reduces the cost to $1/64$.

References for Stable Diffusion v1

See CompVis Stable Diffusion v1 README.
`https://github.com/CompVis/stable-diffusion` (Accessed: 2025-10-26)

See the inference configuration `v1-inference.yaml` for Stable Diffusion v1.5.
`https://huggingface.co/stable-diffusion-v1-5/stable-diffusion-v1-5/blob/main/v1-inference.yaml` (Accessed: 2025-10-26)

See also [5].

What is potential drawbacks of the latent space approach?

The latent space approach replaces initially high-dimensional computation by low-dimensional computation.

What is potential drawbacks of the latent space approach?

The latent space approach replaces initially high-dimensional computation by low-dimensional computation.

Of course, it must make a sacrifice. What is potential drawbacks?

What is potential drawbacks of the latent space approach?

The latent space approach replaces initially high-dimensional computation by low-dimensional computation.

Of course, it must make a sacrifice. What is potential drawbacks?

One drawback is that the (mathematical) image of Decoder_γ cannot cover the whole raw image data space.

What is potential drawbacks of the latent space approach?

The latent space approach replaces initially high-dimensional computation by low-dimensional computation.

Of course, it must make a sacrifice. What is potential drawbacks?

One drawback is that the (mathematical) image of Decoder_γ cannot cover the whole raw image data space.

But, it may not be a practical issue because

What is potential drawbacks of the latent space approach?

The latent space approach replaces initially high-dimensional computation by low-dimensional computation.

Of course, it must make a sacrifice. What is potential drawbacks?

One drawback is that the (mathematical) image of Decoder_γ cannot cover the whole raw image data space.

But, it may not be a practical issue because most pixel tensors are meaningless and not natural images.

What is potential drawbacks of the latent space approach?

The latent space approach replaces initially high-dimensional computation by low-dimensional computation.

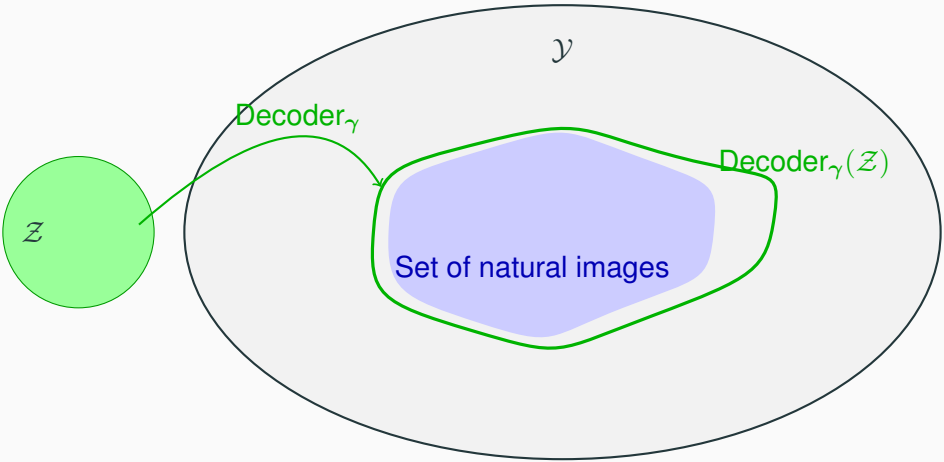
Of course, it must make a sacrifice. What is potential drawbacks?

One drawback is that the (mathematical) image of Decoder_γ cannot cover the whole raw image data space.

But, it may not be a practical issue because most pixel tensors are meaningless and not natural images.

It's sufficient if the image $\text{Decoder}_\gamma(\mathcal{Z}) := \{\text{Decoder}_\gamma(\underline{\boldsymbol{Z}}) \mid \underline{\boldsymbol{Z}} \in \mathcal{Z}\}$ covers **natural images**.

The ideal situation for the VAE decoder



We hope that $\mathrm{Decoder}_\gamma$'s (mathematical) image
$\mathrm{Decoder}_\gamma(\mathcal{Z}) := \{\mathrm{Decoder}_\gamma(\underline{\boldsymbol{Z}}) \mid \underline{\boldsymbol{Z}} \in \mathcal{Z}\}$ sufficiently covers **natural images**.

Why can’t we aim at the entire space? (discrete view)

One might want to construct a scheme to cover the whole raw data space from the latent space. But,

Why can’t we aim at the entire space? (discrete view)

One might want to construct a scheme to cover the whole raw data space from the latent space. But, this is impossible.

Why can't we aim at the entire space? (discrete view)

One might want to construct a scheme to cover the whole raw data space from the latent space. But, this is impossible.

From a discrete perspective, the total number of representable vectors at a fixed precision increases exponentially with the dimension. Therefore, it is trivial that Decoder_γ cannot cover the entire high-dimensional space when $\dim \mathcal{Z} < \dim \mathcal{Y}$.

Why can't we aim at the entire space? (continuous view)

From a continuous perspective, we can roughly say the following.

Theorem (A locally Lipschitz map cannot increase the dimension (informal))

For a set $\mathcal{Z} \subset \mathbb{R}^n$ and a locally Lipschitz map $f : \mathbb{R}^n \to \mathbb{R}^m$, the dimension of $f(\mathcal{Z})$ is no larger than $\mathcal{Z}$. we have

$$\dim(f(\mathcal{Z})) \leq \dim(\mathcal{Z}). \tag{1}$$

Why can't we aim at the entire space? (continuous view)

From a continuous perspective, we can roughly say the following.

Theorem (A locally Lipschitz map cannot increase the dimension (informal))

For a set $\mathcal{Z} \subset \mathbb{R}^n$ and a locally Lipschitz map $f : \mathbb{R}^n \to \mathbb{R}^m$, the dimension of $f(\mathcal{Z})$ is no larger than $\mathcal{Z}$. we have

$$\dim(f(\mathcal{Z})) \leq \dim(\mathcal{Z}). \tag{1}$$

Since a NN is locally Lipschitz, the above informal theorem states that **a NN-based decoder cannot increase the dimension**. If the latent space dimension n is smaller than the raw data space dimension m, we cannot cover the raw data space by the decoder.

What is the dimension of a non-linear subspace?

We can pretend that we've understood the limitation of the latent space method.
But,

What is the dimension of a non-linear subspace?

We can pretend that we've understood the limitation of the latent space method. But, how do we define the dimension? Especially, how about if a set is not a linear subspace?

Answer:

What is the dimension of a non-linear subspace?

We can pretend that we've understood the limitation of the latent space method. But, how do we define the dimension? Especially, how about if a set is not a linear subspace?

Answer: Hausdorff dimension.

Basic idea of the Hausdorff dimension

Consider a 2-dimensional rectangle.

Basic idea of the Hausdorff dimension

Consider a 2-dimensional rectangle.

If you try to measure its 1-dimensional length, it would be infinite. You cannot fill in the plane by a finite length 1-dimensional curve.

Basic idea of the Hausdorff dimension

Consider a 2-dimensional rectangle.

If you try to measure its 1-dimensional length, it would be infinite. You cannot fill in the plane by a finite length 1-dimensional curve.

If you try to measure its 3-dimensional volume, it would be zero. It is, as it were, a infinitely thin shape.

Basic idea of the Hausdorff dimension

Consider a 2-dimensional rectangle.

If you try to measure its 1-dimensional length, it would be infinite. You cannot fill in the plane by a finite length 1-dimensional curve.

If you try to measure its 3-dimensional volume, it would be zero. It is, as it were, a infinitely thin shape.

If you measure its 2-dimensional area, it is a non-zero finite value. This indicates the dimension of the rectangle is 2.

This way, we first define the s-dimensional **measure** of a set, and define the dimensionality as the threshold when the measure turns from infinity to zero.

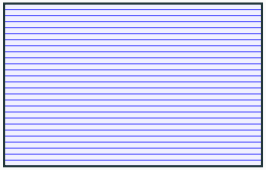
Basic idea of the Hausdorff dimension

Let's consider a 2-dimensional rectangle $R = [0, w] \times [0, h] \times \{0\} \subset \mathbb{R}^3$, where $w, h > 0$.

Basic idea of the Hausdorff dimension

Let's consider a 2-dimensional rectangle $R = [0, w] \times [0, h] \times \{0\} \subset \mathbb{R}^3$, where $w, h > 0$.

Length: $+\infty$

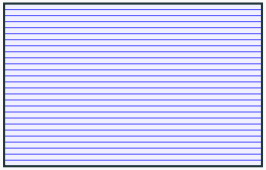


Trying to fill in the set
using N horizontal segments
Total drawn length: $Nw \to \infty$.

Basic idea of the Hausdorff dimension

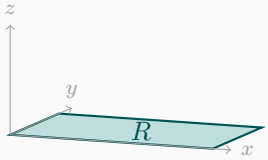
Let's consider a 2-dimensional rectangle $R = [0, w] \times [0, h] \times \{0\} \subset \mathbb{R}^3$, where $w, h > 0$.

Length: $+\infty$



Trying to fill in the set
using N horizontal segments
Total drawn length: $Nw \to \infty$.

Volume: 0

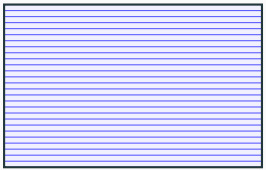


Zero thickness
in the z direction
The rectangle occupies no volume.

Basic idea of the Hausdorff dimension

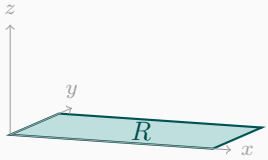
Let's consider a 2-dimensional rectangle $R = [0, w] \times [0, h] \times \{0\} \subset \mathbb{R}^3$, where $w, h > 0$.

Length: $+\infty$



Trying to fill in the set
using N horizontal segments
Total drawn length: $Nw \to \infty$.

Volume: 0



Zero thickness
in the z direction
The rectangle occupies no volume.

Area: $0 < wh < +\infty$



"2-dimensional"

Total area stays equal to wh.

Since the area, the 2-dimensional measure, is non-zero and finite,
we can say that the dimension of the rectangle is 2.
This motivates us to define the general s-dimensional **measure** of a set.

Basic idea of the Hausdorff measure

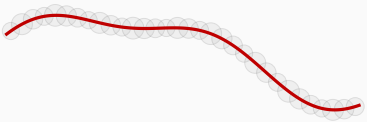
For a set A where a distance function is defined, we define its s-dimensional Hausdorff (outer) measure $\mathcal{H}^s(A)$ according to the following idea.

- Construct an efficient cover of A consisting of infinitely small fragments.
- Price each fragment at the s-dimensional ball volume with the same diameter.
- Sum up the volumes of the fragments.

Basic idea of the Hausdorff measure

For disks of diameter d_i, compare the length cost d_i and the area cost $\pi(d_i/2)^2$.

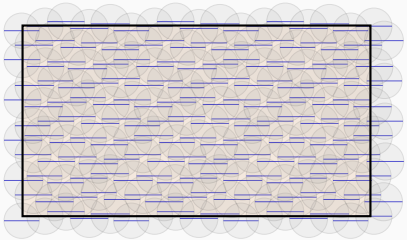
A smooth curve C of finite length



Length cost: $\sum_i d_i \approx \mathrm{Length}(C)$.

Area cost: $\sum_i \pi(d_i/2)^2 \longrightarrow 0$ for finer covers.

A rectangle R of positive area



Length cost: $\sum_i d_i \longrightarrow +\infty$ for finer covers.

Area cost: $\sum_i \pi(d_i/2)^2$ is comparable to $\mathrm{Area}(R)$.

Definition: Hausdorff measure and dimension

Definition (Hausdorff measure (e.g., [2]))

For a non-empty subset A equipped with a distance function $\mathsf{d} : A \times A \to \mathbb{R}_{\geq 0}$, $s \geq 0$, and $\delta > 0$, we define

$$\mathcal{H}^s_\delta(A) := \inf \left\{ \sum_{i=1}^{\infty} \mathsf{BallVol}_s(\operatorname{diam} U_i) \,\middle|\, A \subset \bigcup_{i=1}^{\infty} U_i,\ \operatorname{diam} U_i < \delta \right\}$$

and

Definition: Hausdorff measure and dimension

Definition (Hausdorff measure (e.g., [2]))

For a non-empty subset A equipped with a distance function $\mathsf{d} : A \times A \to \mathbb{R}_{\geq 0}$, $s \geq 0$, and $\delta > 0$, we define

$$\mathcal{H}^s_\delta(A) := \inf \left\{ \sum_{i=1}^{\infty} \mathsf{BallVol}_s(\operatorname{diam} U_i) \,\middle|\, A \subset \bigcup_{i=1}^{\infty} U_i,\ \operatorname{diam} U_i < \delta \right\}$$

and we define the s-dimensional **Hausdorff outer measure** $\mathcal{H}^s(A)$ by $\mathcal{H}^s(A) := \lim_{\delta \downarrow 0} \mathcal{H}^s_\delta(A)$ is called the s-dimensional Hausdorff measure.

Definition: Hausdorff measure and dimension

Definition (Hausdorff measure (e.g., [2]))

For a non-empty subset A equipped with a distance function $\mathsf{d} : A \times A \to \mathbb{R}_{\geq 0}$, $s \geq 0$, and $\delta > 0$, we define

$$\mathcal{H}^s_\delta(A) := \inf \left\{ \sum_{i=1}^{\infty} \mathsf{BallVol}_s(\operatorname{diam} U_i) \,\middle|\, A \subset \bigcup_{i=1}^{\infty} U_i,\ \operatorname{diam} U_i < \delta \right\}$$

and we define the s-dimensional **Hausdorff outer measure** $\mathcal{H}^s(A)$ by $\mathcal{H}^s(A) := \lim_{\delta \downarrow 0} \mathcal{H}^s_\delta(A)$ is called the s-dimensional Hausdorff measure.

Here, $\operatorname{diam} U := \sup\{\mathsf{d}(x, y) \mid x, y \in U\}$, and $\mathsf{BallVol}_s(\delta)$ is the volume of the s-dimensional ball with diameter δ.

Definition: Hausdorff dimension

$\mathsf{BallVol}_s(\delta)$ is given by $\mathsf{BallVol}_s(\delta) = \omega_s \cdot \left(\frac{1}{2}\delta\right)^s$, where ω_s is defined using the Gamma function Γ as $\omega_s := \frac{\pi^{s/2}}{\Gamma\left(\frac{s}{2}+1\right)}$.

Definition: Hausdorff dimension

$\mathrm{BallVol}_s(\delta)$ is given by $\mathrm{BallVol}_s(\delta) = \omega_s \cdot \left(\frac{1}{2}\delta\right)^s$, where ω_s is defined using the Gamma function Γ as $\omega_s := \frac{\pi^{s/2}}{\Gamma\left(\frac{s}{2}+1\right)}$.

When δ is extremely small, $\mathrm{BallVol}_s$ can be regarded as non-increasing function w.r.t. s. Hence, the Hausdorff measure $\mathcal{H}^s(A)$ is also non-increasing for s.

Definition: Hausdorff dimension

$\mathsf{BallVol}_s(\delta)$ is given by $\mathsf{BallVol}_s(\delta) = \omega_s \cdot \left(\frac{1}{2}\delta\right)^s$, where ω_s is defined using the Gamma function Γ as $\omega_s := \frac{\pi^{s/2}}{\Gamma\left(\frac{s}{2}+1\right)}$.

When δ is extremely small, $\mathsf{BallVol}_s$ can be regarded as non-increasing function w.r.t. s. Hence, the Hausdorff measure $\mathcal{H}^s(A)$ is also non-increasing for s.

In fact, it is known that it jumps from $+\infty$ to 0, while it may be non-zero positive at the threshold. That jumping point is Hausdorff dimension.

Definition (Hausdorff dimension (e.g., [2]))

The Hausdorff dimension of a set A equipped with a distance function $\mathsf{d} : A \times A \to \mathbb{R}$ is defined by

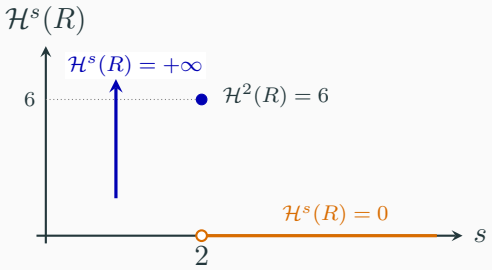
$$\dim_{\mathcal{H}}(A) := \inf\{s \geq 0 : \ \mathcal{H}^s(A) = 0\} = \sup\{s \geq 0 : \ \mathcal{H}^s(A) = \infty\}$$

Remark on definition

Remark

Although it is non-trivial, the Hausdorff dimension of a non-empty subset of a metric space can always be defined and always has a value in $[0, +\infty]$.

The Hausdorff measure of $R = [0, 3] \times [0, 2] \subset \mathbb{R}^2$



Proposition: hypercube dimension

Example (Hausdorff dimension of a hypercube)

The Hausdorff dimension of $A \coloneqq [0,1]^d \subset \mathbb{R}^d$ is given by

$$\dim_{\mathcal{H}}(A) = d. \tag{2}$$

Lipschitz maps

Recall that practical NNs are locally Lipschitz. Let's review the definition.

Definition (Lipschitz map)

For metric spaces (X, d_X) and (Y, d_Y), a map $f : X \to Y$ is said to be **Lipschitz** if there exists a constant $L \geq 0$ such that for all $u, v \in X$,

$$d_Y\big(f(u), f(v)\big) \leq L\, d_X(u, v) \tag{3}$$

holds. The smallest such L is called the Lipschitz constant. In particular, for a map from $(\mathbb{R}^n, \|\cdot\|_2)$ to $(\mathbb{R}^m, \|\cdot\|_2)$, this is

$$\|f(u) - f(v)\|_2 \leq L\, \|u - v\|_2 \tag{4}$$

.

Neighborhoods and local Lipschitz

Definition (Neighborhood in a metric space)

For a metric space (X, d) and a point $x \in X$, a set $U \subset X$ is a **neighborhood** of x if there exists a radius $r > 0$ such that the open ball $B(x, r) \coloneqq \{y \in X : \ d(x, y) < r\}$ satisfies $B(x, r) \subset U$.

Definition (Locally Lipschitz map)

A map $f : X \to Y$ between metric spaces (X, d_X) and (Y, d_Y) is **locally Lipschitz** if for each point $x \in X$, there exists a neighborhood $U_x \subset X$ of x and a constant $L_{U_x} \geq 0$ such that for all $u, v \in U_x$,

$$d_Y\big(f(u), f(v)\big) \leq L_{U_x}\, d_X(u, v) \tag{5}$$

holds.

Example: globally Lipschitz map (sigmoid)

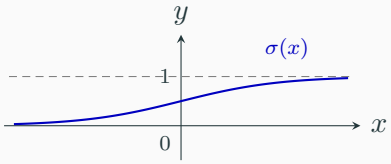
Example (Sigmoid function)

On $(\mathbb{R}, |\cdot|)$,

$$f(x) = \sigma(x) = \frac{1}{1 + e^{-x}}$$

is **globally Lipschitz**. Since $f'(x) = \sigma(x)(1 - \sigma(x))$ and $0 \leq f'(x) \leq 1/4$, we obtain

$$|\sigma(u) - \sigma(v)| \leq \tfrac{1}{4}|u - v|.$$



Example: locally Lipschitz map (x^2)

Example (Square function)

On $(\mathbb{R}, |\cdot|)$,

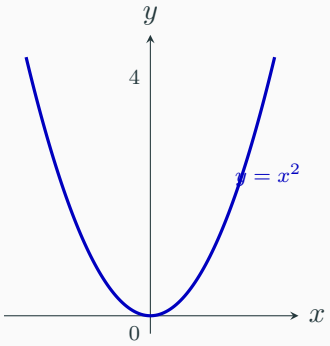
$$f(x) = x^2$$

is **locally Lipschitz**, but not globally.

For $|u|, |v| \le R$,

$$|u^2 - v^2| = |u+v|\,|u-v| \le 2R\,|u-v|.$$

No single finite L works on $\mathbb{R}$, since $|f(t) - f(0)|/|t| = |t| \to \infty$.



Example: not locally Lipschitz ($\sqrt{x}$)

Example (Square-root function)

On $[0, \infty)$,

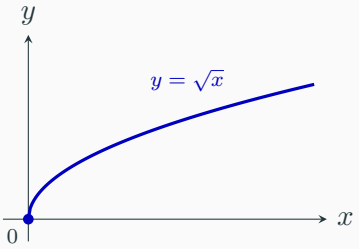
$$f(x) = \sqrt{x}$$

is **not locally Lipschitz at** 0.

Indeed, if f were Lipschitz on a neighborhood of 0 with constant L, then for sufficiently small $t > 0$,

$$\sqrt{t} = |f(t) - f(0)| \leq Lt,$$

so $1/\sqrt{t} \leq L$, which is impossible as $t \downarrow 0$.



Example: globally Lipschitz map (ReLU)

Example (ReLU function)

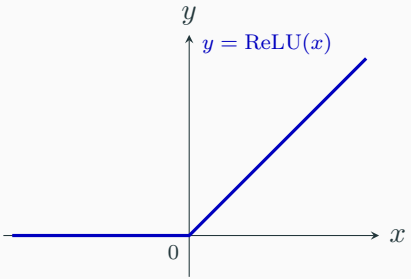
On $(\mathbb{R}, |\cdot|)$,

$$f(x) = \mathrm{ReLU}(x) = \max\{0, x\}$$

is **globally Lipschitz** with $L = 1$. For $u, v \in \mathbb{R}$:

$$u, v \geq 0: \quad |f(u) - f(v)| = |u - v|, \tag{6}$$
$$u, v \leq 0: \quad |f(u) - f(v)| = 0 \leq |u - v|, \tag{7}$$
$$u \geq 0 \geq v: \quad |f(u) - f(v)| = u \leq |u - v|. \tag{8}$$

The case $v \geq 0 \geq u$ is symmetric.



Theorem: dimension non-increase under locally Lipschitz maps

Theorem (Locally Lipschitz map does not increase the Hausdorff dimension (e.g., [2]))

For a set $A \subset \mathbb{R}^n$ and a locally Lipschitz map $f : \mathbb{R}^n \to \mathbb{R}^m$, we have

$$\dim_{\mathcal{H}}(f(A)) \leq \dim_{\mathcal{H}}(A). \tag{9}$$

Theorem: dimension non-increase under locally Lipschitz maps

Theorem (Locally Lipschitz map does not increase the Hausdorff dimension (e.g., [2]))

For a set $A \subset \mathbb{R}^n$ and a locally Lipschitz map $f : \mathbb{R}^n \to \mathbb{R}^m$, we have

$$\dim_{\mathcal{H}}(f(A)) \leq \dim_{\mathcal{H}}(A). \tag{9}$$

Recall that a NN is a locally Lipschitz.

Theorem: dimension non-increase under locally Lipschitz maps

Theorem (Locally Lipschitz map does not increase the Hausdorff dimension (e.g., [2]))

For a set $A \subset \mathbb{R}^n$ and a locally Lipschitz map $f : \mathbb{R}^n \to \mathbb{R}^m$, we have

$$\dim_{\mathcal{H}}(f(A)) \leq \dim_{\mathcal{H}}(A). \tag{9}$$

Recall that a NN is a locally Lipschitz.

As long as we use a latent space method with a NN as a Decoder$_\gamma$, we cannot cover the high-dimensional raw data space.

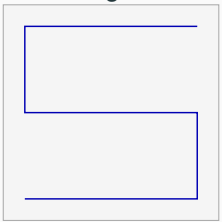
The locally Lipschitz condition is essential

Remark

If a map is NOT locally Lipschitz, then the dimensionality of the image can be larger than that of the domain.

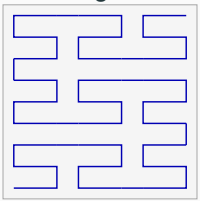
A well-known example are **space-filling curves (Peano curves)**

Stage 1



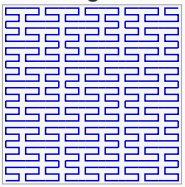
3×3 cells

Stage 2



9×9 cells

Stage 3



27×27 cells

The limiting curve fills the entire square $[0, 1]^2$.

When does the latent-space strategy work? - Answer

Question: When does the latent-space-based pipeline work?

Answer:

When does the latent-space strategy work? - Answer

Question: When does the latent-space-based pipeline work?

Answer: If the subset consisting of “natural” data can be sufficiently covered by a low-dimensional set, the latent-space strategy may work.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. When does the latent space strategy work?

4. How do we adapt various resolutions? - fully convolutional VAE

5. How does a VAE learn? How does it connect the latent space and raw data space?

6. Summary and Next Lecture

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. When does the latent space strategy work?

4. How do we adapt various resolutions? - fully convolutional VAE

5. How does a VAE learn? How does it connect the latent space and raw data space?

6. Summary and Next Lecture

Question: Can we use any architecture for visual data generation?

In visual data generation, we need to adapt various resolutions

In visual data generation, it is common that users specify the output resolutions **at inference phase**.

In visual data generation, we need to adapt various resolutions

In visual data generation, it is common that users specify the output resolutions **at inference phase**.

For example:

- 1024px $\times$ 1024px (square).
- 1344px $\times$ 768px (landscape).
- 768px $\times$ 1344px (portrait).

In visual data generation, we need to adapt various resolutions

In visual data generation, it is common that users specify the output resolutions **at inference phase**.

For example:

- 1024px $\times$ 1024px (square).
- 1344px $\times$ 768px (landscape).
- 768px $\times$ 1344px (portrait).

Hence, we must be able to dynamically construct a NN from one fixed parameter vector so that the output resolution is in line with the user's request.

In visual data generation, we need to adapt various resolutions

In visual data generation, it is common that users specify the output resolutions **at inference phase**.

For example:

- 1024px $\times$ 1024px (square).
- 1344px $\times$ 768px (landscape).
- 768px $\times$ 1344px (portrait).

Hence, we must be able to dynamically construct a NN from one fixed parameter vector so that the output resolution is in line with the user's request.

A commonly used approach is **fully-convolutional neural network**.

Fully convolutional NN can output an visual data of any resolution.

Fully-convolutional refers to the property where all layers consist of **local operations** such as convolution or upsampling.

Fully convolutional NN can output an visual data of any resolution.

Fully-convolutional refers to the property where all layers consist of **local operations** such as convolution or upsampling.

Thus, the network **can accept inputs of variable shape (resolution)**.

Fully convolutional NN can output an visual data of any resolution.

Fully-convolutional refers to the property where all layers consist of **local operations** such as convolution or upsampling.

Thus, the network **can accept inputs of variable shape (resolution)**.

Specifically, the output dimensions (shape) are automatically and uniquely determined according to the input dimensions.

Fully convolutional NN can output an visual data of any resolution.

Fully-convolutional refers to the property where all layers consist of **local operations** such as convolution or upsampling.

Thus, the network **can accept inputs of variable shape (resolution)**.

Specifically, the output dimensions (shape) are automatically and uniquely determined according to the input dimensions.

Hence, by correctly selecting the input shape, the NN can generate images at arbitrary resolutions.

What should we learn in this lecture?

We will look at NN layers used in fully-convolutional neural networks. **Do we need to remember all?**

What should we learn in this lecture?

We will look at NN layers used in fully-convolutional neural networks. **Do we need to remember all?**

Every month, new architectures are released. Remembering every detail of each architecture would not be essential.

What should we learn in this lecture?

We will look at NN layers used in fully-convolutional neural networks. **Do we need to remember all?**

Every month, new architectures are released. Remembering every detail of each architecture would not be essential.

One good learning objective in this section is to confirm for every NN layer:

- The output tensor shape is variable.
- The output tensor shape is determined by the input tensor shape.

What should we learn in this lecture?

We will look at NN layers used in fully-convolutional neural networks. **Do we need to remember all?**

Every month, new architectures are released. Remembering every detail of each architecture would not be essential.

One good learning objective in this section is to confirm for every NN layer:

- The output tensor shape is variable.
- The output tensor shape is determined by the input tensor shape.

The above properties allow the NN to adapt to the requested image resolution.

Variable-length parametric function (1D)

Definition (Variable-Length Parametric Function (1D))

A variable-length parametric function $f_{(\cdot)}$ that takes **one-dimensional tensors (column vectors)** as input/output is a tuple consisting of the following four:

- **Parameter dimension** $d_{\mathrm{param}} \in \mathbb{Z}_{>0}$.
- **Set of acceptable input dimensions** $\mathcal{D}_{\mathrm{in}} \subset \mathbb{Z}_{\geq 0}$.
- **Input/output length mapping** $\mathsf{InLen2OutLen} : \mathcal{D}_{\mathrm{in}} \to \mathbb{Z}_{\geq 0}$.
- A family of maps $\left\{ f_{\boldsymbol{\theta}}^{(L_{\mathrm{in}})} \right\}_{\boldsymbol{\theta} \in \mathbb{R}^{d_{\mathrm{param}}}}$ defined for each $L_{\mathrm{in}} \in \mathcal{D}_{\mathrm{in}}$, such that:

$$f_{\boldsymbol{\theta}}^{(L_{\mathrm{in}})} : \mathbb{R}^{L_{\mathrm{in}}} \to \mathbb{R}^{\mathsf{InLen2OutLen}(L_{\mathrm{in}})}. \tag{10}$$

At this time, the **variable-length parametric function** $f_{(\cdot)}$ itself is defined as:

$$\forall\, \boldsymbol{\theta} \in \mathbb{R}^{d_{\mathrm{param}}},\ \forall\, \boldsymbol{x} \in \bigcup_{L \in \mathcal{D}_{\mathrm{in}}} \mathbb{R}^{L} : \quad f_{\boldsymbol{\theta}}(\boldsymbol{x}) \coloneqq f_{\boldsymbol{\theta}}^{(\dim(\boldsymbol{x}))}(\boldsymbol{x}) \in \mathbb{R}^{\mathsf{InLen2OutLen}(\dim(\boldsymbol{x}))}$$

Mathematical definition: 1D $\star$ operator

Definition (1D $\star$ Operator (1D cross-correlation; basic form))

Let $\boldsymbol{x} \in \mathbb{R}^L$ be a sequence and $K \in \mathbb{Z}_{>0}$ be the kernel length. The index set I_K is given by

$$I_K \coloneqq \begin{cases} \{-r, -r+1, \ldots, 0, \ldots, +r\}, & K = 2r+1 \text{ (odd)}, \\ \{-r+1, -r+2, \ldots, +r\}, & K = 2r \text{ (even)} \end{cases} \tag{12}$$

and the kernel $\boldsymbol{k} = (k_i)_{i \in I_K}$ is arranged in this order (e.g., k_{-1}, k_0, k_{+1} if $K = 3$, and k_0, k_{+1} if $K = 2$). We set zero-padding $x_j = 0$ for $j \notin [0, L-1]_{\mathbb{Z}}$. The $\star$ **operation** $\boldsymbol{y} = \boldsymbol{k} \star \boldsymbol{x}$ is defined by:

$$y_t \coloneqq \sum_{i \in I_K} k_i \, x_{t+i}, \qquad t \in \mathbb{Z} \tag{13}$$

The size of the output of the $\star$ operation

Remark (The size of the output of the $\star$ operation)

As the minimal finite interval excluding t where $y_t = 0$, the **output length** is $L_{\text{out}} = L + K - 1$, and we consider $t \in [0, L_{\text{out}} - 1]_{\mathbb{Z}}$.

Example: 1D $\star$ operation — setup

Example (1D convolution)

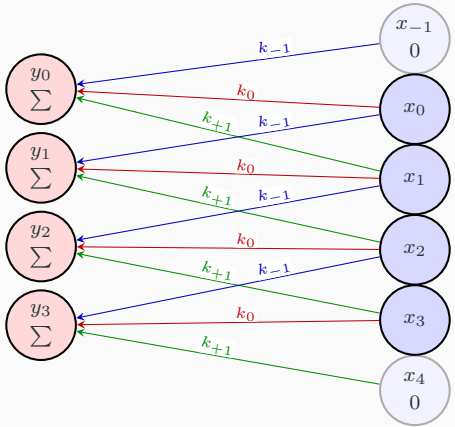
Let the kernel be

$$(k_{-1}, k_0, k_{+1}) = (2, -1, 3), \qquad K = 3.$$

Let the input be

$$\boldsymbol{x} = (x_0, \ldots, x_3) = (1, 2, 0, -1).$$

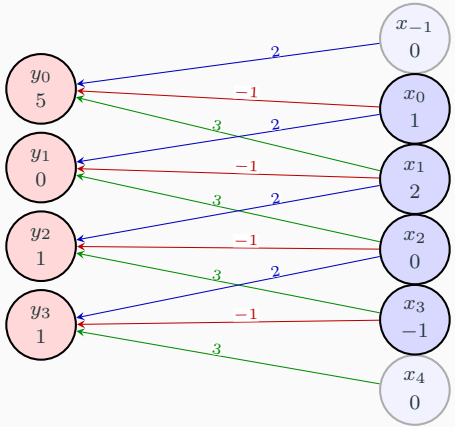
With $L = 4$ and zero-padding, compute $y_t = \sum_{i=-1}^{1} k_i x_{t+i}$ for $t = 0, 1, \ldots, 3$.



Example: 1D $\star$ operation — Answer

$$y_0 = -x_0 + 3x_1 = 5,$$
$$y_1 = 2x_0 - x_1 + 3x_2 = 0,$$
$$y_2 = 2x_1 - x_2 + 3x_3 = 1,$$
$$y_3 = 2x_2 - x_3 = 1.$$

$$(y_0, y_1, y_2, y_3) = (5, 0, 1, 1).$$



Exercise: 1D $\star$ operation — setup

Exercise (1D convolution)

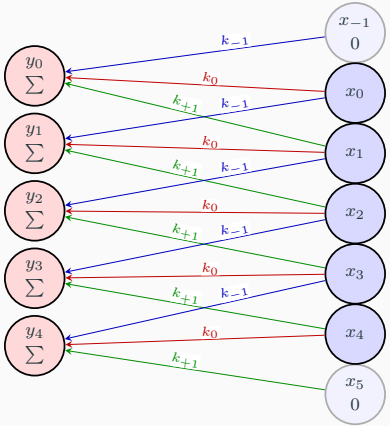
Let the kernel be

$$(k_{-1}, k_0, k_{+1}) = (2, -1, 3), \qquad K = 3.$$

Let the input be

$$\boldsymbol{x} = (x_0, \ldots, x_4) = (3, 0, -2, 1, 4).$$

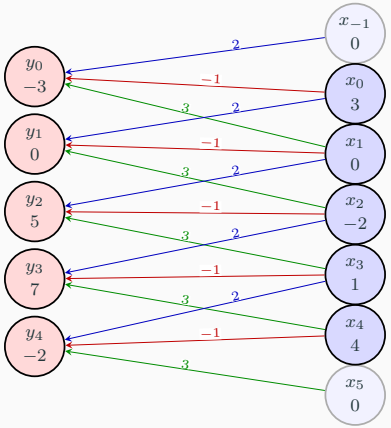
With $L = 5$ and zero-padding, compute $y_t = \sum_{i=-1}^{1} k_i x_{t+i}$ for $t = 0, 1, \ldots, 4$.



Exercise: 1D $\star$ operation — Answer

$$
\begin{aligned}
y_0 &= -x_0 + 3x_1 = -3, \\
y_1 &= 2x_0 - x_1 + 3x_2 = 0, \\
y_2 &= 2x_1 - x_2 + 3x_3 = 5, \\
y_3 &= 2x_2 - x_3 + 3x_4 = 7, \\
y_4 &= 2x_3 - x_4 = -2.
\end{aligned}
$$

$$(y_0, y_1, y_2, y_3, y_4) = (-3, 0, 5, 7, -2).$$



Convolutional layer can achieve adaptive output length

- If you want to output a vector with length 4 output, you can input a vector with length 4.
- If you want to output a vector with length 5 output, you can input a vector with length 5.

This means:

Convolutional layer can achieve adaptive output length

- If you want to output a vector with length 4 output, you can input a vector with length 4.
- If you want to output a vector with length 5 output, you can input a vector with length 5.

This means:

- The output tensor shape is variable.
- The output tensor shape is determined by the input tensor shape.

Remark: convolution sign convention

Remark (Sign Convention for "Convolution" in This Lecture)

The term "convolution" as used in this lecture refers to **cross-correlation**, which is common in implementation. That is, for a 1D and 2 D signal

$$(y \text{ by } \star)_t \coloneqq \sum_{i \in I_K} k_i \, x_{\,t+i}, \quad (Y \text{ by } \star)_{u,v} \coloneqq \sum_{i \in I_{K_H}} \sum_{j \in I_{K_W}} K_{i,j} \, X_{\,u+i,\,v+j}, \tag{14}$$

it is defined by (with zero-padding). The standard **convolution** in mathematics and signal processing involves flipping the kernel

$$(y \text{ by } *)_t = \sum_{i} k_i \, x_{\,t-i}, \qquad (Y \text{ by } *)_{u,v} = \sum_{i,j} K_{i,j} \, X_{\,u-i,\,v-j} \tag{15}$$

thus, the sign of the indices is reversed. The symbol $\star$ in this lecture is used to explicitly denote **cross-correlation** as aligned with implementation.

Remark: on “n-dimensional convolution”

Remark (On the Term “n-dimensional Convolution”)

Conventionally, the term **n-dimensional convolution** is used, but strictly speaking, it often refers to the **order** of the tensor, i.e., the number of spatial axes. In these notes, we follow convention and use the terms 1D/2D.

1D convolutional layer: parameters and shapes

Definition (1D Convolutional Layer (with stride/padding/dilation, multi-channel, activation; variable-length parametric function by $\star$))

Hyperparameters: We are given stride $S \in \mathbb{Z}_{>0}$, padding $P \in \mathbb{Z}_{\geq 0}$, dilation $D \in \mathbb{Z}_{>0}$, input/output channel counts $m, n \in \mathbb{Z}_{>0}$, and kernel length $K \in \mathbb{Z}_{>0}$. The index set I_K is as in (12). The learnable parameters are:

$$\boldsymbol{k}^{[c,r]} = (k_i^{[c,r]})_{i \in I_K}, \qquad b^{[c]} \in \mathbb{R} \tag{16}$$

Acceptable input lengths: $\mathcal{D}_{\text{in}} = \mathbb{Z}_{\geq 0}$.

Input/output length mapping: For an input length $L \in \mathbb{Z}_{\geq 0}$,

$$\mathsf{InLen2OutLen}(L) \;\coloneqq\; \left\lfloor \frac{L + 2P - D\,(K-1) - 1}{S} \right\rfloor + 1 \;\in\; \mathbb{Z}_{\geq 0}. \tag{17}$$

1D convolutional layer: forward and activation

Definition (continued)

Definition of map family: For any $L \in \mathcal{D}_{\text{in}}$, let the input be $\boldsymbol{X} \in \mathbb{R}^{m \times L}$ and the output be $\boldsymbol{Y} \in \mathbb{R}^{n \times \mathsf{InLen2OutLen}(L)}$. We denote the sequence for each input channel as $\boldsymbol{x}^{[r]} \in \mathbb{R}^{L}$ $(r = 1, \ldots, m)$ and for each output channel as $\boldsymbol{y}^{[c]} \in \mathbb{R}^{\mathsf{InLen2OutLen}(L)}$ $(c = 1, \ldots, n)$. We define the zero-padded vector $\tilde{\boldsymbol{x}}^{[r]} \in \mathbb{R}^{L+2P}$ as:

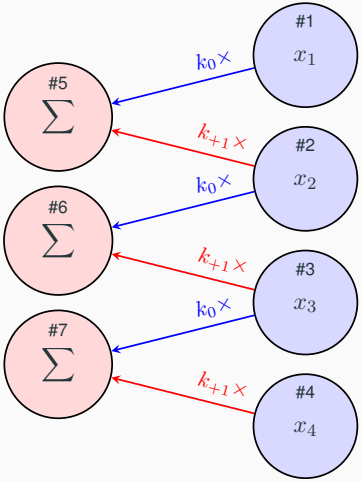
$$\tilde{x}_t^{[r]} = \begin{cases} x_{t-P}^{[r]}, & t \in [0, L + 2P - 1]_{\mathbb{Z}}, \\ 0, & \text{otherwise} \end{cases} \tag{18}$$

At this time, for $t \in [0, \mathsf{InLen2OutLen}(L) - 1]_{\mathbb{Z}}$:

$$y_t^{[c]} \coloneqq \sum_{r=1}^{m} \sum_{i \in I_K} k_i^{[c,r]} \, \tilde{x}_{tS+iD}^{[r]} \; + \; b^{[c]} \quad (c = 1, \ldots, n), \tag{19}$$

$$\widehat{\boldsymbol{Y}} \coloneqq \mathsf{Act}(\boldsymbol{Y}) \quad \text{(Activation Act is applied element-wise).} \tag{20}$$

1D convolutional layer. 1ch→1ch, $L = 4, K = 2, S = 1$. $L_{\text{out}} = 3$



1D convolutional layer. 1ch→1ch, $L = 3, K = 2, S = 1$**.** $L_{\text{out}} = 2$

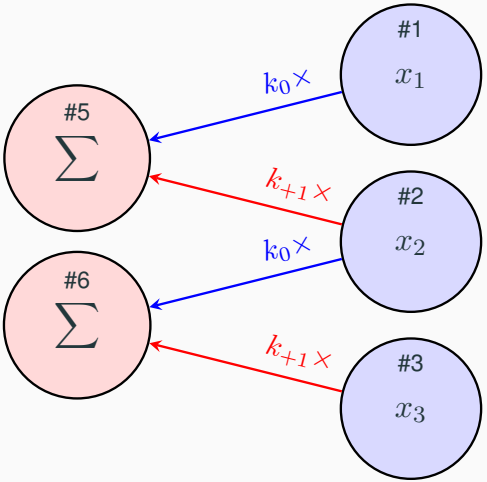


Figure: 1D 1ch→2ch, $L = 4, K = 2, S = 1$

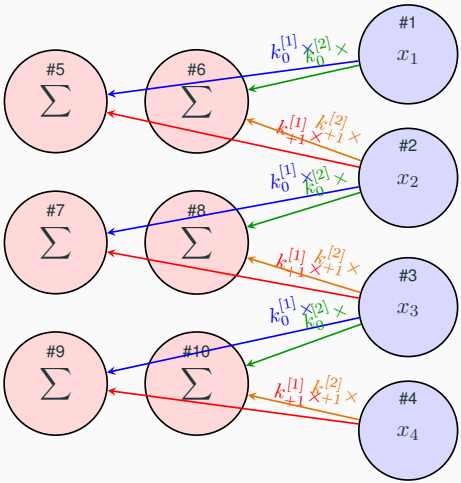


Figure 2: 1D 1ch→2ch, $L = 4, K = 2, S = 1$ ($\star$ operation).

Figure: 1D 1ch→2ch, $L = 3, K = 2, S = 1$

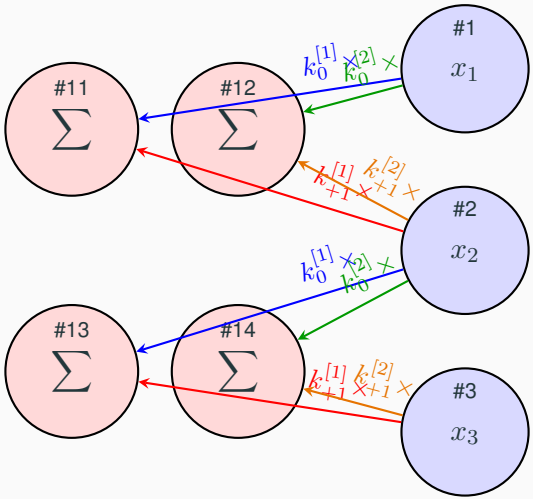


Figure 3: 1D 1ch→2ch, $L = 3, K = 2, S = 1$ ($\star$ operation).

Figure: 1D 2ch→2ch, $L = 4, K = 2, S = 1$

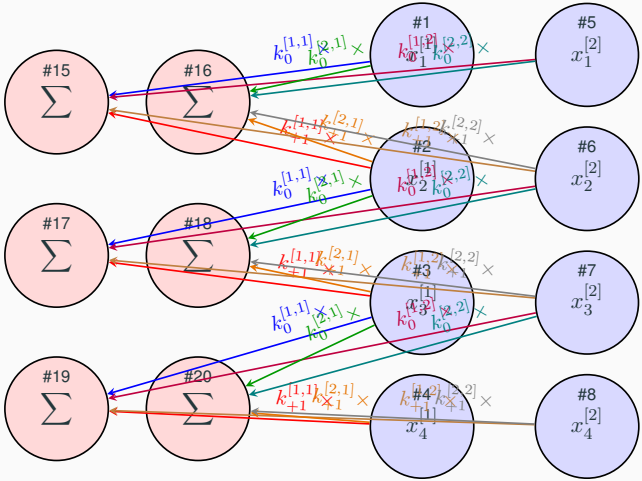


Figure: 1D 2ch→2ch, $L = 3, K = 2, S = 1$

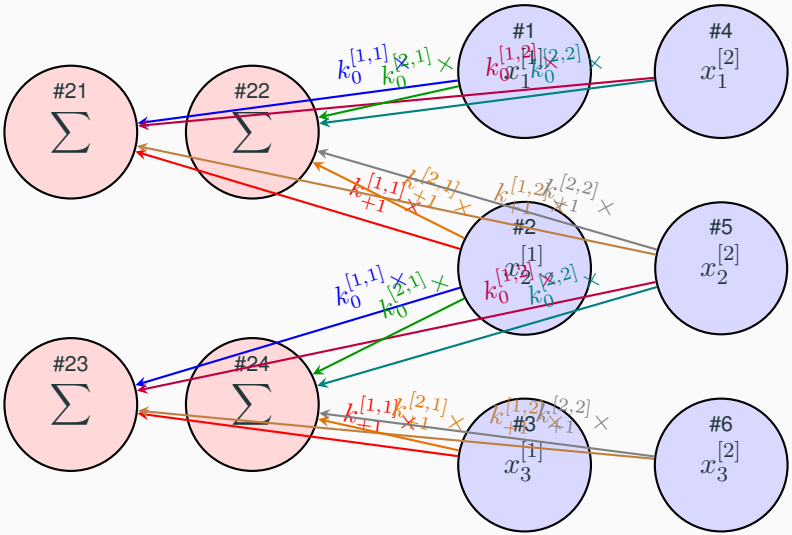


Figure 4: 1D 2ch →2ch, $L = 3, K = 2, S = 1$ (+ operation)

Figure: 1D $K = 3, S = 1, P = 1, D = 1$

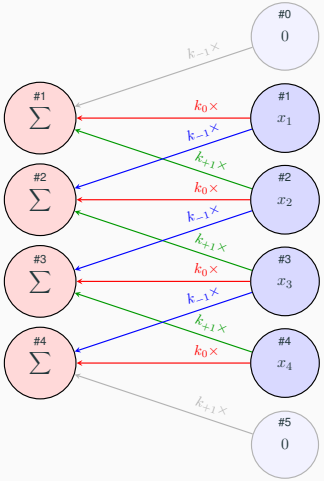


Figure 5: 1D 1ch→1ch, $L = 4, K = 3, S = 1, P = 1, D = 1$ ($\star$ operation; $L_{\text{out}} = 4$).

Figure: 1D $K = 3, S = 2, P = 1, D = 1$

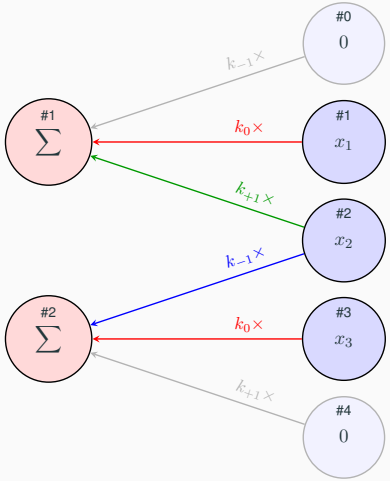


Figure 6: 1D 1ch→1ch, $L = 3, K = 3, S = 2, P = 1, D = 1$ ($\star$ operation; $L_{\text{out}} = 2$).

Figure: 1D $K = 3, S = 1, P = 1, D = 2$

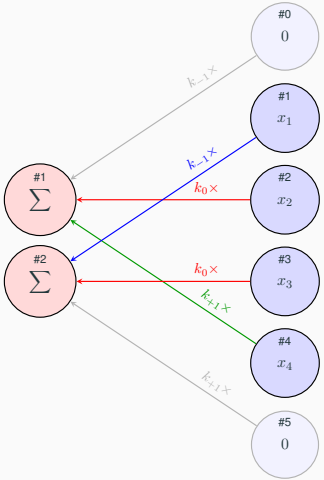


Figure 7: 1D 1ch→1ch, $L = 4, K = 3, S = 1, P = 1, D = 2$ ($\star$ operation; $L_{\text{out}} = 2$).

1D nearest-neighbor upscaler

Definition (1D Nearest-Neighbor Upscaler (Variable-Length))

Given an upscale factor $s \in \mathbb{Z}_{\geq 2}$. Parameters are empty ($d_{\text{param}} = 0$), the acceptable set is $\mathcal{D}_{\text{in}} = \mathbb{Z}_{\geq 0}$, and the input/output length mapping is defined by:

$$\mathsf{InLen2OutLen}(L) \coloneqq sL \tag{21}$$

For any L and input $\boldsymbol{x} \in \mathbb{R}^L$, it is defined by:

$$\left(f^{(L)}(\boldsymbol{x})\right)_t \;\coloneqq\; x_{\lfloor t/s \rfloor}, \qquad t \in [0, sL-1]_{\mathbb{Z}} \tag{22}$$

(where $\lfloor \cdot \rfloor$ is the floor function).

Figure: 1D NN upscaler example

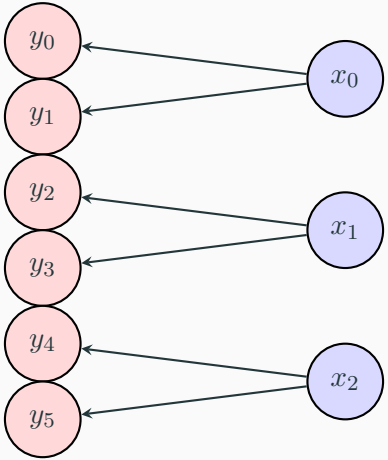


Figure 8: 1D Nearest-Neighbor Upscaler (example with $s = 2$, $L = 3$).

1D group normalization

Definition (Group Normalization (1D; Variable-Length))

Given the number of channels $c \in \mathbb{Z}_{>0}$, and the number of partitions g such that $g \mid c$. The input is $\boldsymbol{F} \in \mathbb{R}^{c \times L}$ (each row is $\boldsymbol{f}^{[i]} \in \mathbb{R}^L$). For each partition $G_k = \{(k-1)\frac{c}{g} + 1, \ldots, k\frac{c}{g}\}$:

$$\mu_k \coloneqq \frac{1}{|G_k|\, L} \sum_{i \in G_k} \sum_{t=1}^{L} f_t^{[i]}, \qquad \sigma_k^2 \coloneqq \frac{1}{|G_k|\, L} \sum_{i \in G_k} \sum_{t=1}^{L} \big(f_t^{[i]} - \mu_k\big)^2. \tag{23}$$

Using learnable coefficients $\gamma_i, \beta_i \in \mathbb{R}$ ($i = 1, \ldots, c$) and $\varepsilon > 0$:

$$\widehat{f}_t^{[i]} \coloneqq \gamma_i \frac{f_t^{[i]} - \mu_{k(i)}}{\sqrt{\sigma_{k(i)}^2 + \varepsilon}} + \beta_i, \qquad i \in G_{k(i)},\ t = 1, \ldots, L. \tag{24}$$

Group normalization operation is variable-length

The group normalization operation has a number of learnable parameters independent of L, and the output dimension of the function is determined depending on L, thus it provides a variable-length parametric function.

Variable-shape parametric function (2D)

Definition (Variable-shape Parametric Function (2D))

A variable-shape parametric function $F_{(\cdot)}$ that takes **two-dimensional tensors** as input/output consists of $d_{\text{param}} \in \mathbb{Z}_{>0}$, acceptable sets $\mathcal{H}, \mathcal{W} \subset \mathbb{Z}_{\geq 0}$, an input/output mapping $\text{HW2HW}_{\text{out}} : \mathcal{H} \times \mathcal{W} \to \mathbb{Z}_{\geq 0} \times \mathbb{Z}_{\geq 0}$, and a family for each (H, W):

$$F_{\boldsymbol{\theta}}^{(H,W)} : \mathbb{R}^{H \times W} \to \mathbb{R}^{H_{\text{out}}(H,W) \times W_{\text{out}}(H,W)} \tag{25}$$

The function itself is defined as $F_{\boldsymbol{\theta}}(\boldsymbol{X}) \coloneqq F_{\boldsymbol{\theta}}^{(\dim_1(\boldsymbol{X}), \dim_2(\boldsymbol{X}))}(\boldsymbol{X})$.

Mathematical definition: 2D $\star$ operator

Definition (2D $\star$ Operator (2D cross-correlation; basic form))

Let the input be $\boldsymbol{X} \in \mathbb{R}^{H \times W}$, and the filter sizes be $K_H, K_W \in \mathbb{Z}_{>0}$. The index sets

$$I_{K_H},\ I_{K_W} \text{ are defined similarly to (12).} \tag{26}$$

Let the kernel be $\boldsymbol{K} = (K_{i,j})_{i \in I_{K_H},\, j \in I_{K_W}}$. We set zero-padding $X_{p,q} = 0$ for $(p, q) \notin [0, H-1]_{\mathbb{Z}} \times [0, W-1]_{\mathbb{Z}}$, and define $\boldsymbol{Y} = \boldsymbol{K} \star \boldsymbol{X}$ as:

$$Y_{u,v} \coloneqq \sum_{i \in I_{K_H}} \sum_{j \in I_{K_W}} K_{i,j}\, X_{u+i,\, v+j}, \qquad (u, v) \in \mathbb{Z}^2 \tag{27}$$

Restricting to the minimal rectangular region that can be non-zero, $H_{\text{out}} = H + K_H - 1$, $W_{\text{out}} = W + K_W - 1$, and we consider $u \in [0, H_{\text{out}} - 1]_{\mathbb{Z}}$, $v \in [0, W_{\text{out}} - 1]_{\mathbb{Z}}$.

Example: 2D $\star$ operation

Example (2D cross-correlation with zero-padding)

Let

$$\boldsymbol{X} = \begin{bmatrix} 1 & 0 \\ 2 & -1 \end{bmatrix} \in \mathbb{R}^{2\times 2}, \qquad \boldsymbol{K} = \begin{bmatrix} 1 & 2 & 0 \\ 3 & -1 & 1 \\ 2 & 0 & -2 \end{bmatrix} \in \mathbb{R}^{3\times 3}.$$

The kernel indices are $I_{K_H} = I_{K_W} = \{-1, 0, +1\}$; $K_{-1,-1} = 1$ and $K_{+1,+1} = -2$. Set $X_{p,q} = 0$ whenever $(p, q) \notin \{0, 1\}^2$.

The full output is indexed by $a, b \in \{0, 1, 2, 3\}$, with

$$Y_{a,b} \coloneqq (\boldsymbol{K} \star \boldsymbol{X})_{a-1,b-1} = \sum_{i=-1}^{1} \sum_{j=-1}^{1} K_{i,j} X_{a-1+i,b-1+j}.$$

Thus $\boldsymbol{Y} \in \mathbb{R}^{4\times 4}$. Compute all 16 entries of $\boldsymbol{Y}$.

Example: 2D $\star$ operation — Answer

For $a, b \in \{0, 1, 2, 3\}$, let

$\boldsymbol{P}^{(a,b)} := \left(X_{a-1+i,b-1+j}\right)_{i,j\in\{-1,0,1\}}$, $\quad Y_{a,b} = \sum_{i=-1}^{1} \sum_{j=-1}^{1} K_{i,j} X_{a-1+i,b-1+j}$.

Example computation

$(a, b) = (0, 0)$

0 / 1	0 / 2	0 / 0
0 / 3	0 / −1	0 / 1
0 / 2	0 / 0	1 / −2

blue: $\boldsymbol{P}^{(a,b)}$

red: $\boldsymbol{K}$

$(a, b) = (1, 2)$

0 / 1	0 / 2	0 / 0
1 / 3	0 / −1	0 / 1
2 / 2	−1 / 0	0 / −2

Complete output $\boldsymbol{Y}$

	0	1	2	3
0	−2	0	2	0
1	−3	1	7	−2
2	2	−1	8	−3
3	0	4	0	−1

$$\boldsymbol{Y} = \begin{bmatrix} -2 & 0 & 2 & 0 \\ -3 & 1 & 7 & -2 \\ 2 & -1 & 8 & -3 \\ 0 & 4 & 0 & -1 \end{bmatrix}.$$

Exercise: 2D $\star$ operation — Setup

Exercise (2D cross-correlation with a 3×3 kernel)

Let

$$\boldsymbol{X} = \begin{bmatrix} 0 & 1 & 2 \\ -1 & 3 & 1 \end{bmatrix} \in \mathbb{R}^{2\times 3}, \qquad \boldsymbol{K} = \begin{bmatrix} 2 & 0 & -1 \\ 1 & -1 & 2 \\ 0 & 1 & 1 \end{bmatrix} \in \mathbb{R}^{3\times 3}.$$

Take $I_{K_H} = I_{K_W} = \{-1, 0, +1\}$ and set $X_{p,q} = 0$ outside $\{0, 1\} \times \{0, 1, 2\}$.

Define

$$Y_{a,b} \coloneqq (\boldsymbol{K} \star \boldsymbol{X})_{a-1,b-1} = \sum_{i=-1}^{1} \sum_{j=-1}^{1} K_{i,j} X_{a-1+i,b-1+j}$$

for $a \in \{0, 1, 2, 3\}$ and $b \in \{0, 1, 2, 3, 4\}$.

Compute $\boldsymbol{Y} \in \mathbb{R}^{4\times 5}$, including $Y_{2,2}$ and $Y_{3,4}$.

Exercise: 2D $\star$ operation — Answer

The output is computed with zero-padding and the same kernel at every position.

$$
\begin{aligned}
Y_{2,2} &= 2(0) + 0(1) - 1(2) \\
&\quad + 1(-1) - 1(3) + 2(1) \\
&\quad + 0(0) + 1(0) + 1(0) \\
&= -4,
\end{aligned}
$$

$$
Y_{3,4} = 2X_{1,2} = 2(1) = 2.
$$

Complete output Y

	0	1	2	3	4
0	0	1	3	2	0
1	−1	4	7	0	2
2	−2	6	−4	4	5
3	1	−3	−3	6	2

2D convolutional layer: parameters and output size

Definition (2D Convolutional Layer (with stride/padding/dilation, multi-channel, activation))

Let the input be $\underline{\boldsymbol{X}} \in \mathbb{R}^{m \times H \times W}$ ($\boldsymbol{X}^{[r]}$ is the feature map of input channel r), and the output be $\underline{\boldsymbol{Y}} \in \mathbb{R}^{n \times H_{\text{out}} \times W_{\text{out}}}$. We are given $S_H, S_W \in \mathbb{Z}_{>0}$, $P_H, P_W \in \mathbb{Z}_{\geq 0}$, $D_H, D_W \in \mathbb{Z}_{>0}$. The learnable parameters are:

$$\boldsymbol{K}^{[c,r]} = (K_{i,j}^{[c,r]})_{i \in I_{K_H}, j \in I_{K_W}}, \qquad b^{[c]} \in \mathbb{R}. \tag{28}$$

We define the zero-padding $\tilde{X}_{p,q}^{[r]}$ as:

$$\tilde{X}_{p,q}^{[r]} = \begin{cases} X_{p-P_H, q-P_W}^{[r]}, & p \in [0, H + 2P_H - 1]_{\mathbb{Z}},\ q \in [0, W + 2P_W - 1]_{\mathbb{Z}}, \\ 0, & \text{otherwise} \end{cases} \tag{29}$$

2D convolutional layer: forward and activation

Definition (continued)

The output spatial size is:

$$H_{\text{out}} \coloneqq \left\lfloor \frac{H + 2P_H - D_H\,(K_H - 1) - 1}{S_H} \right\rfloor + 1, \tag{30}$$

$$W_{\text{out}} \coloneqq \left\lfloor \frac{W + 2P_W - D_W\,(K_W - 1) - 1}{S_W} \right\rfloor + 1. \tag{31}$$

For each c and (p, q):

$$Y_{p,q}^{[c]} \coloneqq \sum_{r=1}^{m} \sum_{i \in I_{K_H}} \sum_{j \in I_{K_W}} K_{i,j}^{[c,r]}\; \tilde{X}_{pS_H + iD_H,\, qS_W + jD_W}^{[r]} \;+\; b^{[c]}, \tag{32}$$

$$\underline{\widehat{\boldsymbol{Y}}} \coloneqq \mathsf{Act}(\underline{\boldsymbol{Y}})\,. \tag{33}$$

Figure: 2D 1ch→1ch, stride 1

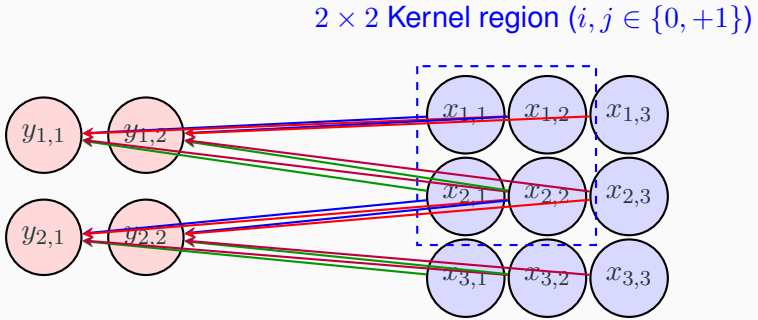


Figure 9: Example of 2D 1ch→1ch (3×3 input, 2×2 kernel, stride 1; $\star$ operation). $H_{\text{out}} = W_{\text{out}} = 2$ (Eq. (30),(31)).

Figure: 2D $m$ch$\rightarrow$ $n$ch schematic

For input $\underline{\boldsymbol{X}} \in \mathbb{R}^{m \times H \times W}$ and output $\underline{\boldsymbol{Y}} \in \mathbb{R}^{n \times H_{\text{out}} \times W_{\text{out}}}$,

$$Y_{p,q}^{[c]} = \sum_{r=1}^{m} \sum_{i \in I_{K_H}} \sum_{j \in I_{K_W}} K_{i,j}^{[c,r]} \, \tilde{X}_{pS_H+iD_H, qS_W+jD_W}^{[r]} + b^{[c]}. \tag{34}$$

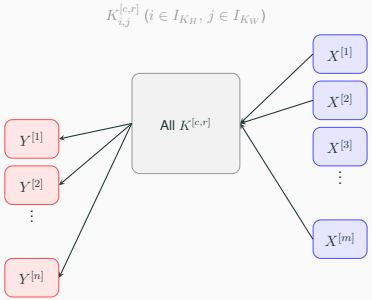


Figure 10: Schematic of 2D $m$ch$\rightarrow$ $n$ch (Eq. (30) – (32), (34); $\star$ operation).

2D nearest-neighbor upscaler

Definition (2D Nearest-Neighbor Upscaler (Variable-Length))

Given an upscale factor $s \in \mathbb{Z}_{\geq 2}$. $d_{\text{param}} = 0$, the acceptable sets are $\mathcal{H} = \mathcal{W} = \mathbb{Z}_{\geq 0}$, and the input/output mapping is $(H, W) \mapsto (sH, sW)$. For an input $\boldsymbol{X} \in \mathbb{R}^{H \times W}$,

$$\left(F^{(H,W)}(\boldsymbol{X})\right)_{sp+a,\ sw+b} \coloneqq X_{p,q}, \quad a, b \in [0, s-1]_{\mathbb{Z}},\ \ (p, q) \in [0, H-1]_{\mathbb{Z}} \times [0, W-1]_{\mathbb{Z}}. \tag{35}$$

2D group normalization

Definition (Group Normalization (2D; Variable-Length))

With the same settings as Definition 19, the **layer output is defined** according to any non-negative integers H, W. That is, when giving $\widehat{\underline{\boldsymbol{F}}}$ for $\underline{\boldsymbol{F}} \in \mathbb{R}^{c \times H \times W}$ using Eq. (23) and (24), we define a family of variable-length parametric functions $\{\widehat{\underline{\boldsymbol{F}}}\}$ with $d_{\text{param}} = 2c$ (the output spatial size is the same as the input, (H, W)).

The spatial attention layer is also a variable shape layer

Definition (Spatial self-attention)

Let $H, W, C \in \mathbb{Z}_{>0}$ and $\underline{\boldsymbol{X}} \in \mathbb{R}^{H \times W \times C}$. For each spatial position $(i, j) \in [1, H]_{\mathbb{Z}} \times [1, W]_{\mathbb{Z}}$, define the column vector $\boldsymbol{x}_{i,j} := \begin{bmatrix} \underline{\boldsymbol{X}}_{i,j,1} & \cdots & \underline{\boldsymbol{X}}_{i,j,C} \end{bmatrix}^\top \in \mathbb{R}^C$.

Let $\boldsymbol{W}_{\mathsf{Q}}, \boldsymbol{W}_{\mathsf{K}}, \boldsymbol{W}_{\mathsf{V}}, \boldsymbol{W}_{\mathsf{O}} \in \mathbb{R}^{C \times C}$ and $\boldsymbol{b}_{\mathsf{Q}}, \boldsymbol{b}_{\mathsf{K}}, \boldsymbol{b}_{\mathsf{V}}, \boldsymbol{b}_{\mathsf{O}} \in \mathbb{R}^C$ be learned parameters. Define

$$\boldsymbol{q}_{i,j} = \boldsymbol{W}_{\mathsf{Q}} \boldsymbol{x}_{i,j} + \boldsymbol{b}_{\mathsf{Q}}, \quad \boldsymbol{k}_{i,j} = \boldsymbol{W}_{\mathsf{K}} \boldsymbol{x}_{i,j} + \boldsymbol{b}_{\mathsf{K}}, \quad \boldsymbol{v}_{i,j} = \boldsymbol{W}_{\mathsf{V}} \boldsymbol{x}_{i,j} + \boldsymbol{b}_{\mathsf{V}}.$$

For spatial positions (i, j) and (i', j'), define the **attention weight**

$$a_{(i,j),(i',j')} := \frac{\exp\left(\boldsymbol{q}_{i,j}^\top \boldsymbol{k}_{i',j'} / \sqrt{C}\right)}{\sum_{r=1}^{H} \sum_{s=1}^{W} \exp\left(\boldsymbol{q}_{i,j}^\top \boldsymbol{k}_{r,s} / \sqrt{C}\right)}.$$

The spatial attention layer is also a variable shape layer

Definition (Spatial self-attention (cont.))

The output vector at position (i, j) is
$\boldsymbol{y}_{i,j} = \boldsymbol{x}_{i,j} + \boldsymbol{W}_O \left(\sum_{i'=1}^{H} \sum_{j'=1}^{W} a_{(i,j),(i',j')} \boldsymbol{v}_{i',j'} \right) + \boldsymbol{b}_O$.

The output tensor $\underline{\boldsymbol{Y}} \in \mathbb{R}^{H \times W \times C}$ is defined by $\underline{\boldsymbol{Y}}_{i,j,c} = (\boldsymbol{y}_{i,j})_c$.

Really?

Exercise (The spatial attention layer is a variable shape layer)

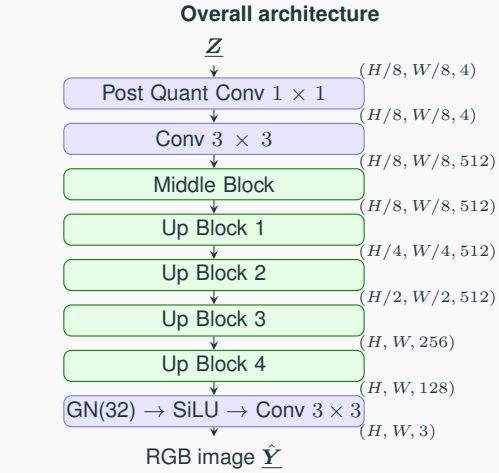
For the spatial attention layer, confirm that

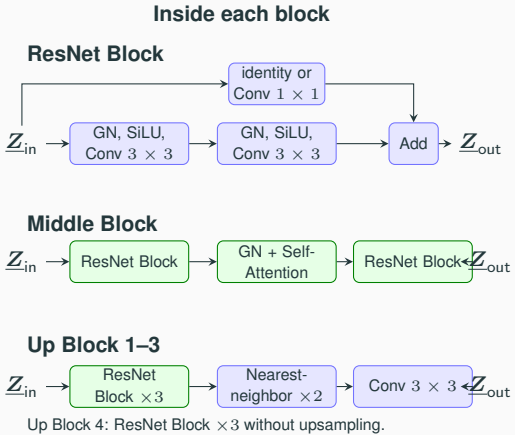
- The output height/width shape is variable.
- The output tensor height/width is determined by the input tensor shape.

Answer

- When the input tensor shape is (H, W, C), then the output tensor shape is also (H, W, C).
- The shape of parameters $\boldsymbol{W}_{\mathsf{Q}}, \boldsymbol{W}_{\mathsf{K}}, \boldsymbol{W}_{\mathsf{V}}, \boldsymbol{W}_{\mathsf{O}} \in \mathbb{R}^{C \times C}$ and $\boldsymbol{b}_{\mathsf{Q}}, \boldsymbol{b}_{\mathsf{K}}, \boldsymbol{b}_{\mathsf{V}}, \boldsymbol{b}_{\mathsf{O}} \in \mathbb{R}^{C}$ do not depend on H or W.

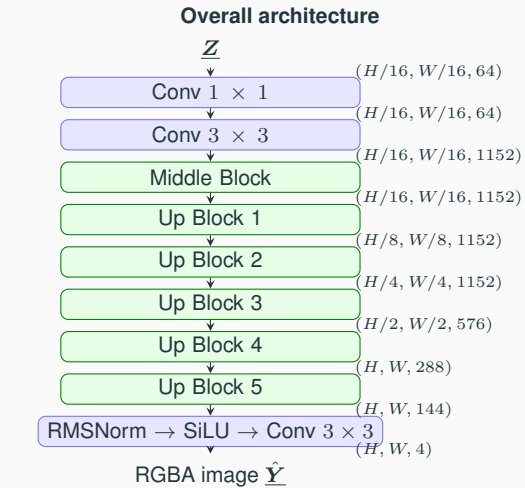
Decoder in Stable Diffusion 1.5

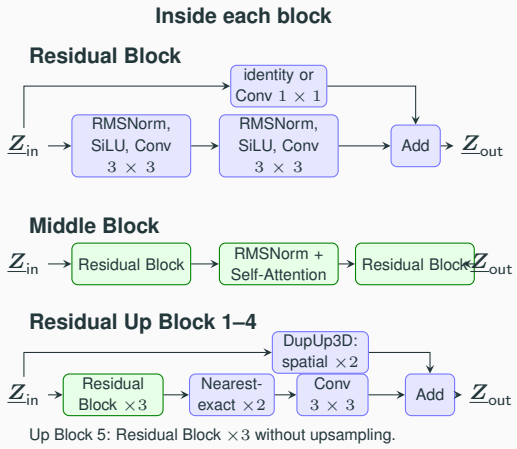




Arrow labels: (height, width, channels); $H, W \in 8\mathbb{Z}_{>0}$. Source: CompVis, model.py (Decoder, ResnetBlock, AttnBlock, Upsample).

Decoder in Qwen-Image-2.1





Arrow labels: (height, width, channels); $H, W \in 16\mathbb{Z}_{>0}$; the singleton time axis is omitted. Source: Diffusers, autoencoder_kl_qwenimage21.py.

Fully convolutional network - answer

Question: Can we use any architecture for visual data generation?

Answer: In visual data generation, we need to adapt to the resolution users specify. A common approach to achieve this is to use a **fully convolutional neural network**. Here, fully convolutional neural network consists of variable-shape layers such as convolutional layers, upscaler, etc.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. When does the latent space strategy work?

4. How do we adapt various resolutions? - fully convolutional VAE

5. How does a VAE learn? How does it connect the latent space and raw data space?

6. Summary and Next Lecture

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. When does the latent space strategy work?

4. How do we adapt various resolutions? - fully convolutional VAE

5. How does a VAE learn? How does it connect the latent space and raw data space?

6. Summary and Next Lecture

Question: How do we learn VAE in the image generation pipeline? Is it the same as the original VAE?

Autoencoder: matching the “natural” subset and the image of the decoder.

We hope the image of the $\text{Decoder}_{\theta_{\text{Dec}}} : \mathcal{Z} \to \mathcal{Y}$ can cover natural image subset in the high-dimensional raw image space $\mathcal{Y}$.

Autoencoder: matching the “natural” subset and the image of the decoder.

We hope the image of the $\text{Decoder}_{\theta_{\text{Dec}}} : \mathcal{Z} \to \mathcal{Y}$ can cover natural image subset in the high-dimensional raw image space $\mathcal{Y}$.

Hence, when training $\text{Decoder}_{\theta_{\text{Dec}}}$, we also train a parametric map $\text{Encoder}_{\theta_{\text{Enc}}} : \mathcal{Y} \to \mathcal{Z}$ working like an “inverse” map of $\text{Decoder}_{\theta_{\text{Dec}}}$.

Autoencoder: matching the “natural” subset and the image of the decoder.

We hope the image of the $\text{Decoder}_{\boldsymbol{\theta}_{\text{Dec}}} : \mathcal{Z} \to \mathcal{Y}$ can cover natural image subset in the high-dimensional raw image space $\mathcal{Y}$.

Hence, when training $\text{Decoder}_{\boldsymbol{\theta}_{\text{Dec}}}$, we also train a parametric map $\text{Encoder}_{\boldsymbol{\theta}_{\text{Enc}}} : \mathcal{Y} \to \mathcal{Z}$ working like an “inverse” map of $\text{Decoder}_{\boldsymbol{\theta}_{\text{Dec}}}$.

The training objective is to achieve the following

$$\text{Decoder}_{\boldsymbol{\theta}_{\text{Dec}}}(\text{Encoder}_{\boldsymbol{\theta}_{\text{Enc}}}(\underline{\boldsymbol{Y}})) \approx \underline{\boldsymbol{Y}}$$

for $\underline{\boldsymbol{Y}}$ sampled from the natural image distribution.

Autoencoder: matching the "natural" subset and the image of the decoder.

We hope the image of the $\text{Decoder}_{\boldsymbol{\theta}_{\text{Dec}}} : \mathcal{Z} \to \mathcal{Y}$ can cover natural image subset in the high-dimensional raw image space $\mathcal{Y}$.

Hence, when training $\text{Decoder}_{\boldsymbol{\theta}_{\text{Dec}}}$, we also train a parametric map $\text{Encoder}_{\boldsymbol{\theta}_{\text{Enc}}} : \mathcal{Y} \to \mathcal{Z}$ working like an "inverse" map of $\text{Decoder}_{\boldsymbol{\theta}_{\text{Dec}}}$.

The training objective is to achieve the following

$$\text{Decoder}_{\boldsymbol{\theta}_{\text{Dec}}}(\text{Encoder}_{\boldsymbol{\theta}_{\text{Enc}}}(\underline{\boldsymbol{Y}})) \approx \underline{\boldsymbol{Y}}$$

for $\underline{\boldsymbol{Y}}$ sampled from the natural image distribution.

If the image of $\text{Decoder}_{\boldsymbol{\theta}_{\text{Dec}}}$ does not cover natural images, then the above approximation cannot be achieved for a natural image $\underline{\boldsymbol{Y}}$.

The loss function of an autoencoder (AE)

The AE's training objective is to achieve the following

$$\text{Decoder}_{\boldsymbol{\theta}_{\text{Dec}}}(\text{Encoder}_{\boldsymbol{\theta}_{\text{Enc}}}(\underline{\boldsymbol{Y}})) \approx \underline{\boldsymbol{Y}}$$

for $\underline{\boldsymbol{Y}}$ sampled from the natural image distribution.

The loss function of an autoencoder (AE)

The AE's training objective is to achieve the following

$$\text{Decoder}_{\boldsymbol{\theta}_{\text{Dec}}}(\text{Encoder}_{\boldsymbol{\theta}_{\text{Enc}}}(\underline{\boldsymbol{Y}})) \approx \underline{\boldsymbol{Y}}$$

for $\underline{\boldsymbol{Y}}$ sampled from the natural image distribution.

When we use the ℓ_1 loss as a reconstruction error function, we can formalize the above approximation on the dataset $(\underline{\boldsymbol{Y}}^{(1)}, \underline{\boldsymbol{Y}}^{(2)}, ..., \underline{\boldsymbol{Y}}^{(B)})$ by the following minimization problem:

$$\underset{\boldsymbol{\theta}_{\text{Dec}}, \boldsymbol{\theta}_{\text{Enc}}}{\text{minimize}} \quad \frac{1}{B} \sum_{b=1}^{B} \left\| \text{Decoder}_{\boldsymbol{\theta}_{\text{Dec}}}\left(\text{Encoder}_{\boldsymbol{\theta}_{\text{Enc}}}(\underline{\boldsymbol{Y}}^{(b)})\right) - \underline{\boldsymbol{Y}}^{(b)} \right\|_1,$$

where $\|\cdot\|$ returns the absolute sum of all the elements in the tensor.

Why we avoid using a vanilla AE in the image generation pipeline?

A vanilla AE may learn a good one-to-one correspondence between the latent space $\mathcal{Z}$ and natural images.

However,

Why we avoid using a vanilla AE in the image generation pipeline?

A vanilla AE may learn a good one-to-one correspondence between the latent space $\mathcal{Z}$ and natural images.

However, minimizing AE's loss function does not care the smoothness of the correspondence.

Why we avoid using a vanilla AE in the image generation pipeline?

A vanilla AE may learn a good one-to-one correspondence between the latent space $\mathcal{Z}$ and natural images.

However, minimizing AE's loss function does not care the smoothness of the correspondence.

Specifically, consider a small perturbation $\underline{\epsilon}$. $\text{Decoder}_{\boldsymbol{\theta}_{\text{Dec}}}(\underline{\boldsymbol{Z}})$ and $\text{Decoder}_{\boldsymbol{\theta}_{\text{Dec}}}(\underline{\boldsymbol{Z}} + \underline{\boldsymbol{\epsilon}})$ may be totally different.

Why we avoid using a vanilla AE in the image generation pipeline?

A vanilla AE may learn a good one-to-one correspondence between the latent space $\mathcal{Z}$ and natural images.

However, minimizing AE's loss function does not care the smoothness of the correspondence.

Specifically, consider a small perturbation $\underline{\epsilon}$. $\text{Decoder}_{\theta_{\text{Dec}}}(\underline{\mathcal{Z}})$ and $\text{Decoder}_{\theta_{\text{Dec}}}(\underline{\mathcal{Z}} + \underline{\epsilon})$ may be totally different.

If the $\text{Decoder}_{\theta_{\text{Dec}}}$ is too sensitive to perturbation in $\mathcal{Z}$, it makes the training of other components significantly difficult.

Why we avoid using a vanilla AE in the image generation pipeline?

A vanilla AE may learn a good one-to-one correspondence between the latent space $\mathcal{Z}$ and natural images.

However, minimizing AE's loss function does not care the smoothness of the correspondence.

Specifically, consider a small perturbation $\underline{\epsilon}$. $\text{Decoder}_{\boldsymbol{\theta}_{\text{Dec}}}(\underline{\boldsymbol{Z}})$ and $\text{Decoder}_{\boldsymbol{\theta}_{\text{Dec}}}(\underline{\boldsymbol{Z}} + \underline{\epsilon})$ may be totally different.

If the $\text{Decoder}_{\boldsymbol{\theta}_{\text{Dec}}}$ is too sensitive to perturbation in $\mathcal{Z}$, it makes the training of other components significantly difficult.

VAE's idea: to minimize the reconstruction error with perturbation.

Variational autoencoder (VAE) - reconstruction part

For each natural image $\underline{\boldsymbol{Y}}^{(b)}$, we aim to achieve the following:

$$\text{Decoder}_{\boldsymbol{\theta}_{\text{Dec}}}(\text{Encoder}_{\boldsymbol{\theta}_{\text{Enc}}}(\underline{\boldsymbol{Y}}^{(b)}) + \underline{\boldsymbol{\sigma}}_{\boldsymbol{\theta}_{\text{Enc}}}(\underline{\boldsymbol{Y}}^{(b)}) \odot \underline{\boldsymbol{\epsilon}}^{(b)}) \approx \underline{\boldsymbol{Y}}^{(b)}$$

for $\underline{\boldsymbol{Y}}^{(b)}$ sampled from the natural image distribution and $\underline{\boldsymbol{\epsilon}}^{(b)}$ is sampled Gaussian noise.

Here, $\underline{\boldsymbol{\sigma}}_{\boldsymbol{\theta}_{\text{Enc}}} : \mathcal{Y} \to \mathcal{Z}$ is a NN that controls how much perturbation we allow for each pixel and each natural image. The NN $\underline{\boldsymbol{\sigma}}_{\boldsymbol{\theta}_{\text{Enc}}}$ shares most parameters and architecture with $\text{Encoder}_{\boldsymbol{\theta}_{\text{Enc}}}$.

Variational autoencoder (VAE) - reconstruction part

For each natural image $\underline{\boldsymbol{Y}}^{(b)}$, we aim to achieve the following:

$$\text{Decoder}_{\boldsymbol{\theta}_{\text{Dec}}}(\text{Encoder}_{\boldsymbol{\theta}_{\text{Enc}}}(\underline{\boldsymbol{Y}}^{(b)}) + \underline{\boldsymbol{\sigma}}_{\boldsymbol{\theta}_{\text{Enc}}}(\underline{\boldsymbol{Y}}^{(b)}) \odot \underline{\boldsymbol{\epsilon}}^{(b)}) \approx \underline{\boldsymbol{Y}}^{(b)}$$

for $\underline{\boldsymbol{Y}}^{(b)}$ sampled from the natural image distribution and $\underline{\boldsymbol{\epsilon}}^{(b)}$ is sampled Gaussian noise.

Here, $\underline{\boldsymbol{\sigma}}_{\boldsymbol{\theta}_{\text{Enc}}} : \mathcal{Y} \to \mathcal{Z}$ is a NN that controls how much perturbation we allow for each pixel and each natural image. The NN $\underline{\boldsymbol{\sigma}}_{\boldsymbol{\theta}_{\text{Enc}}}$ shares most parameters and architecture with $\text{Encoder}_{\boldsymbol{\theta}_{\text{Enc}}}$.

If the above approximation is achieved, then small perturbation in $\mathcal{Z}$ does not change the output image significantly.

The reconstruction loss of VAE

The VAE's training objective is to achieve the following

$$\text{Decoder}_{\boldsymbol{\theta}_{\text{Dec}}}(\text{Encoder}_{\boldsymbol{\theta}_{\text{Enc}}}(\underline{\boldsymbol{Y}}^{(b)}) + \underline{\boldsymbol{\sigma}}_{\boldsymbol{\theta}_{\text{Enc}}}(\underline{\boldsymbol{Y}}^{(b)}) \odot \underline{\boldsymbol{\epsilon}}^{(b)}) \approx \underline{\boldsymbol{Y}}^{(b)}.$$

The reconstruction loss of VAE

The VAE's training objective is to achieve the following

$$\text{Decoder}_{\boldsymbol{\theta}_{\text{Dec}}}(\text{Encoder}_{\boldsymbol{\theta}_{\text{Enc}}}(\underline{\boldsymbol{Y}}^{(b)}) + \underline{\boldsymbol{\sigma}}_{\boldsymbol{\theta}_{\text{Enc}}}(\underline{\boldsymbol{Y}}^{(b)}) \odot \underline{\boldsymbol{\epsilon}}^{(b)}) \approx \underline{\boldsymbol{Y}}^{(b)}.$$

For the natural image data point i.i.d. Gaussian noise sample $(\underline{\boldsymbol{Y}}^{(b)}, \underline{\boldsymbol{\epsilon}}^{(b)})$, the reconstruction loss of the VAE for each is given as follows:

$$\left\| \text{Decoder}_{\boldsymbol{\theta}_{\text{Dec}}}\left(\text{Encoder}_{\boldsymbol{\theta}_{\text{Enc}}}(\underline{\boldsymbol{Y}}^{(b)}) + \underline{\boldsymbol{\sigma}}_{\boldsymbol{\theta}_{\text{Enc}}}(\underline{\boldsymbol{Y}}^{(b)}) \odot \underline{\boldsymbol{\epsilon}}^{(b)}\right) - \underline{\boldsymbol{Y}}^{(b)} \right\|_1$$
$$+ \frac{1}{2}\left\| \text{Encoder}_{\boldsymbol{\theta}_{\text{Enc}}}(\underline{\boldsymbol{Y}}^{(b)}) \right\|_2^2 + \frac{1}{2}\left\| \left(\underline{\boldsymbol{\sigma}}_{\boldsymbol{\theta}_{\text{Enc}}}(\underline{\boldsymbol{Y}}^{(b)})\right)^2 - \mathbf{1} - 2\log \underline{\boldsymbol{\sigma}}_{\boldsymbol{\theta}_{\text{Enc}}}(\underline{\boldsymbol{Y}}^{(b)}) \right\|_1$$

The second term regularizes the latent representation so it does not diverge.

The third term regularizes $\underline{\boldsymbol{\sigma}}_{\boldsymbol{\theta}_{\text{Enc}}}(\underline{\boldsymbol{Y}}^{(b)})$ so it is around 1. Otherwise, it would shrink to zero and minimize the perturbation in the first term.

Rewrite the loss function of the VAE

For an image tensor $\underline{\boldsymbol{Y}}^{(b)}$ and latent noise $\underline{\boldsymbol{\epsilon}}$, define the following:

- $\underline{\hat{\boldsymbol{Z}}}^{(b)}_{\boldsymbol{\theta}_{\text{Enc}},\underline{\boldsymbol{Y}}^{(b)}} := \text{Encoder}_{\boldsymbol{\theta}_{\text{Enc}}}(\underline{\boldsymbol{Y}}^{(b)})$
- $\underline{\tilde{\boldsymbol{Z}}}^{(b)}_{\boldsymbol{\theta}_{\text{Enc}},\underline{\boldsymbol{Y}}^{(b)},\underline{\boldsymbol{\epsilon}}^{(b)}} := \underline{\hat{\boldsymbol{Z}}}^{(b)}_{\boldsymbol{\theta}_{\text{Enc}},\underline{\boldsymbol{Y}}^{(b)}} + \underline{\boldsymbol{\sigma}}_{\boldsymbol{\theta}_{\text{Enc}}}(\underline{\boldsymbol{Y}}^{(b)}) \odot \underline{\boldsymbol{\epsilon}}^{(b)}$
- $\underline{\hat{\boldsymbol{Y}}}^{(b)}_{\boldsymbol{\theta}_{\text{Dec}},\boldsymbol{\theta}_{\text{Enc}},\underline{\boldsymbol{Y}}^{(b)},\underline{\boldsymbol{\epsilon}}^{(b)}} := \text{Decoder}_{\boldsymbol{\theta}_{\text{Dec}}}\left(\underline{\tilde{\boldsymbol{Z}}}^{(b)}_{\boldsymbol{\theta}_{\text{Enc}},\underline{\boldsymbol{Y}}^{(b)},\underline{\boldsymbol{\epsilon}}^{(b)}}\right)$

Rewrite the loss function of the VAE

For an image tensor $\underline{\boldsymbol{Y}}^{(b)}$ and latent noise $\underline{\boldsymbol{\epsilon}}$, define the following:

- $\underline{\hat{\boldsymbol{Z}}}^{(b)}_{\boldsymbol{\theta}_{\text{Enc}},\underline{\boldsymbol{Y}}^{(b)}} := \text{Encoder}_{\boldsymbol{\theta}_{\text{Enc}}}(\underline{\boldsymbol{Y}}^{(b)})$
- $\underline{\tilde{\boldsymbol{Z}}}^{(b)}_{\boldsymbol{\theta}_{\text{Enc}},\underline{\boldsymbol{Y}}^{(b)},\underline{\boldsymbol{\epsilon}}^{(b)}} := \underline{\hat{\boldsymbol{Z}}}^{(b)}_{\boldsymbol{\theta}_{\text{Enc}},\underline{\boldsymbol{Y}}^{(b)}} + \underline{\boldsymbol{\sigma}}_{\boldsymbol{\theta}_{\text{Enc}}}(\underline{\boldsymbol{Y}}^{(b)}) \odot \underline{\boldsymbol{\epsilon}}^{(b)}$
- $\underline{\hat{\boldsymbol{Y}}}^{(b)}_{\boldsymbol{\theta}_{\text{Dec}},\boldsymbol{\theta}_{\text{Enc}},\underline{\boldsymbol{Y}}^{(b)},\underline{\boldsymbol{\epsilon}}^{(b)}} := \text{Decoder}_{\boldsymbol{\theta}_{\text{Dec}}}\big(\underline{\tilde{\boldsymbol{Z}}}^{(b)}_{\boldsymbol{\theta}_{\text{Enc}},\underline{\boldsymbol{Y}}^{(b)},\underline{\boldsymbol{\epsilon}}^{(b)}}\big)$

The reconstruction loss of the VAE can be written as follows:

$$\left\| \underline{\hat{\boldsymbol{Y}}}^{(b)}_{\boldsymbol{\theta}_{\text{Dec}},\boldsymbol{\theta}_{\text{Enc}},\underline{\boldsymbol{Y}}^{(b)},\underline{\boldsymbol{\epsilon}}^{(b)}} - \underline{\boldsymbol{Y}}^{(b)} \right\|_1 + \text{Reg}\big(\underline{\hat{\boldsymbol{Z}}}^{(b)}_{\boldsymbol{\theta}_{\text{Enc}},\underline{\boldsymbol{Y}}^{(b)}}, \underline{\boldsymbol{\sigma}}_{\boldsymbol{\theta}_{\text{Enc}}}(\underline{\boldsymbol{Y}}^{(b)})\big),$$

where Reg corresponds to the second and third regularization terms in the previous page.

VAE in visual data generation is not a vanilla VAE.

The VAE loss function in the previous page is almost following the original VAE paper [4].

However, in real image generation [5], we are not satisfied with the original VAE.

Additional key tricks [1] are

- Penalizing the perceptual reconstruction error by LPIPS [7].
- Introducing adversarial learning [3].

Penalizing perceptual reconstruction error

The ℓ_1 loss $\|\cdot\|$ only measures the difference in the pixel space.

We also

Penalizing perceptual reconstruction error

The ℓ_1 loss $\|\cdot\|$ only measures the difference in the pixel space.

We also want to measure the perceptual difference.

Idea of LPIPS: Measuring errors inside a fixed image recognition NN. We expect an image recognition NN can extract the perceptual meaning of the given image.

Penalizing the perceptual reconstruction error by LPIPS

LPIPS (Learned Perceptual Image Patch Similarity) [7] uses VGG16 [6], a image recognition neural network to measure semantic dissimilarity between two images.

Penalizing the perceptual reconstruction error by LPIPS

LPIPS (Learned Perceptual Image Patch Similarity) [7] uses VGG16 [6], a image recognition neural network to measure semantic dissimilarity between two images.

Since we do not retrain the VGG16 in VAE training, we can simply regard the LPIPS loss as a fixed function. The loss is now as follows:

$$\left\| \underline{\hat{\boldsymbol{Y}}}^{(b)}_{\boldsymbol{\theta}_{\text{Dec}}, \boldsymbol{\theta}_{\text{Enc}}, \underline{\boldsymbol{Y}}^{(b)}, \underline{\boldsymbol{\epsilon}}^{(b)}} - \underline{\boldsymbol{Y}}^{(b)} \right\|_1 + \text{LPIPS}\left(\underline{\hat{\boldsymbol{Y}}}^{(b)}_{\boldsymbol{\theta}_{\text{Dec}}, \boldsymbol{\theta}_{\text{Enc}}, \underline{\boldsymbol{Y}}^{(b)}, \underline{\boldsymbol{\epsilon}}^{(b)}}, \underline{\boldsymbol{Y}}^{(b)} \right) + \text{Reg}\left(\underline{\hat{\boldsymbol{Z}}}^{(b)}_{\boldsymbol{\theta}_{\text{Enc}}, \underline{\boldsymbol{Y}}^{(b)}}, \underline{\boldsymbol{\sigma}}_{\boldsymbol{\theta}_{\text{Enc}}}(\underline{\boldsymbol{Y}}^{(b)}) \right)$$

Here, $\text{LPIPS}\left(\underline{\hat{\boldsymbol{Y}}}^{(b)}_{\boldsymbol{\theta}_{\text{Dec}}, \boldsymbol{\theta}_{\text{Enc}}, \underline{\boldsymbol{Y}}^{(b)}, \underline{\boldsymbol{\epsilon}}^{(b)}}, \underline{\boldsymbol{Y}}^{(b)} \right)$ measures how much different the VGG16 representations of $\underline{\hat{\boldsymbol{Y}}}^{(b)}_{\boldsymbol{\theta}_{\text{Dec}}, \boldsymbol{\theta}_{\text{Enc}}, \underline{\boldsymbol{Y}}^{(b)}, \underline{\boldsymbol{\epsilon}}^{(b)}}$ and $\underline{\boldsymbol{Y}}^{(b)}$.

Adversarial learning

We even want to improve the image quality so that it looks natural.

Adversarial learning

We even want to improve the image quality so that it looks natural.

Idea of adversarial learning: We penalize if $\text{Decoder}_{\theta_{\text{Dec}}}$ cannot deceive a NN $\text{Discriminator}_{\theta_{\text{Disc}}}$.

Discriminator learning

We prepare a fully convolutional NN Discriminator$_{\boldsymbol{\theta}_{\text{Disc}}} : \mathcal{Y} \to \mathbb{R}^{W_{\text{Disc}}, H_{\text{Disc}}}$.

Discriminator learning

We prepare a fully convolutional NN Discriminator$_{\boldsymbol{\theta}_{\text{Disc}}} : \mathcal{Y} \to \mathbb{R}^{W_{\text{Disc}}, H_{\text{Disc}}}$.

We hope that Discriminator$_{\boldsymbol{\theta}_{\text{Disc}}}$ detects a natural image. Specifically,

Discriminator learning

We prepare a fully convolutional NN Discriminator$_{\boldsymbol{\theta}_{\text{Disc}}} : \mathcal{Y} \to \mathbb{R}^{W_{\text{Disc}}, H_{\text{Disc}}}$.

We hope that Discriminator$_{\boldsymbol{\theta}_{\text{Disc}}}$ detects a natural image. Specifically, we hope that Discriminator$_{\boldsymbol{\theta}_{\text{Disc}}}$ returns a low score for reconstructed image $\underline{\hat{\boldsymbol{Y}}}^{(b)}_{\boldsymbol{\theta}_{\text{Dec}}, \boldsymbol{\theta}_{\text{Enc}}, \underline{\boldsymbol{Y}}^{(b)}, \underline{\boldsymbol{\epsilon}}^{(b)}}$ and a high score for a natural image $\underline{\boldsymbol{Y}}^{(b)}$.

Hence, we optimize $\boldsymbol{\theta}_{\text{Disc}}$ by minimizing the following loss function:

$$\left\| [\mathbf{1} - \text{Discriminator}_{\boldsymbol{\theta}_{\text{Disc}}}(\underline{\boldsymbol{Y}}^{(b)})]_+ + [\mathbf{1} + \text{Discriminator}_{\boldsymbol{\theta}_{\text{Disc}}}(\underline{\hat{\boldsymbol{Y}}}^{(b)}_{\boldsymbol{\theta}_{\text{Dec}}, \boldsymbol{\theta}_{\text{Enc}}, \underline{\boldsymbol{Y}}^{(b)}, \underline{\boldsymbol{\epsilon}}^{(b)}})]_+ \right\|_1.$$

Adversarial loss

On the other hand, we hope that $\text{Decoder}_{\theta_{\text{Dec}}}$ generates a high-quality image so that it deceives $\text{Discriminator}_{\theta_{\text{Disc}}}$.

Adversarial loss

On the other hand, we hope that Decoder$_{\boldsymbol{\theta}_{\text{Dec}}}$ generates a high-quality image so that it deceives Discriminator$_{\boldsymbol{\theta}_{\text{Disc}}}$.

Hence, we add the following term to the VAE loss:

$$-\mathbf{1}^{\mathsf{T}}\text{Discriminator}_{\boldsymbol{\theta}_{\text{Disc}}}(\underline{\hat{\boldsymbol{Y}}}^{(b)}_{\boldsymbol{\theta}_{\text{Dec}},\boldsymbol{\theta}_{\text{Enc}},\underline{\boldsymbol{Y}}^{(b)},\underline{\boldsymbol{\epsilon}}^{(b)}}).$$

VAE in modern visual data generation is more than VAE

In the first half of the training, we minimize the following loss w.r.t. $\boldsymbol{\theta}_{\text{Dec}}, \boldsymbol{\theta}_{\text{Enc}}$:

$$\left\| \underline{\hat{\boldsymbol{Y}}}^{(b)}_{\boldsymbol{\theta}_{\text{Dec}}, \boldsymbol{\theta}_{\text{Enc}}, \underline{\boldsymbol{Y}}^{(b)}, \underline{\boldsymbol{\epsilon}}^{(b)}} - \underline{\boldsymbol{Y}}^{(b)} \right\|_1 + \text{LPIPS}\left(\underline{\hat{\boldsymbol{Y}}}^{(b)}_{\boldsymbol{\theta}_{\text{Dec}}, \boldsymbol{\theta}_{\text{Enc}}, \underline{\boldsymbol{Y}}^{(b)}, \underline{\boldsymbol{\epsilon}}^{(b)}}, \underline{\boldsymbol{Y}}^{(b)} \right) + \text{Reg}\left(\underline{\hat{\boldsymbol{Z}}}^{(b)}_{\boldsymbol{\theta}_{\text{Enc}}, \underline{\boldsymbol{Y}}^{(b)}}, \underline{\boldsymbol{\sigma}}_{\boldsymbol{\theta}_{\text{Enc}}}(\underline{\boldsymbol{Y}}^{(b)}) \right),$$

which consists of

- ℓ_1 reconstruction error on raw data space with latent space perturbation
- Perceptual reconstruction error on image recognition NN space
- Terms penalizing too large latent representations and too small perturbation.

VAE in modern visual data generation is more than VAE

In the first half of the training, we minimize the following loss w.r.t. $\boldsymbol{\theta}_{\mathsf{Dec}}, \boldsymbol{\theta}_{\mathsf{Enc}}$:

$$\left\| \underline{\hat{\boldsymbol{Y}}}^{(b)}_{\boldsymbol{\theta}_{\mathsf{Dec}}, \boldsymbol{\theta}_{\mathsf{Enc}}, \underline{\boldsymbol{Y}}^{(b)}, \underline{\boldsymbol{\epsilon}}^{(b)}} - \underline{\boldsymbol{Y}}^{(b)} \right\|_1 + \mathsf{LPIPS}\big(\underline{\hat{\boldsymbol{Y}}}^{(b)}_{\boldsymbol{\theta}_{\mathsf{Dec}}, \boldsymbol{\theta}_{\mathsf{Enc}}, \underline{\boldsymbol{Y}}^{(b)}, \underline{\boldsymbol{\epsilon}}^{(b)}}, \underline{\boldsymbol{Y}}^{(b)}\big) + \mathsf{Reg}\big(\hat{\underline{\boldsymbol{Z}}}^{(b)}_{\boldsymbol{\theta}_{\mathsf{Enc}}, \underline{\boldsymbol{Y}}^{(b)}}, \underline{\boldsymbol{\sigma}}_{\boldsymbol{\theta}_{\mathsf{Enc}}}(\underline{\boldsymbol{Y}}^{(b)})\big),$$

which consists of

- ℓ_1 reconstruction error on raw data space with latent space perturbation
- Perceptual reconstruction error on image recognition NN space
- Terms penalizing too large latent representations and too small perturbation.

In the latter half, we add the following adversarial loss:

$$-\lambda \mathbf{1}^{\mathsf{T}} \mathsf{Discriminator}_{\boldsymbol{\theta}_{\mathsf{Disc}}}(\underline{\hat{\boldsymbol{Y}}}^{(b)}_{\boldsymbol{\theta}_{\mathsf{Dec}}, \boldsymbol{\theta}_{\mathsf{Enc}}, \underline{\boldsymbol{Y}}^{(b)}, \underline{\boldsymbol{\epsilon}}^{(b)}}).$$

VAE in modern visual data generation is more than VAE

In the first half of the training, we minimize the following loss w.r.t. $\boldsymbol{\theta}_{\mathsf{Dec}}, \boldsymbol{\theta}_{\mathsf{Enc}}$:

$$\left\| \hat{\underline{\boldsymbol{Y}}}^{(b)}_{\boldsymbol{\theta}_{\mathsf{Dec}}, \boldsymbol{\theta}_{\mathsf{Enc}}, \underline{\boldsymbol{Y}}^{(b)}, \underline{\boldsymbol{\epsilon}}^{(b)}} - \underline{\boldsymbol{Y}}^{(b)} \right\|_1 + \mathsf{LPIPS}\big(\hat{\underline{\boldsymbol{Y}}}^{(b)}_{\boldsymbol{\theta}_{\mathsf{Dec}}, \boldsymbol{\theta}_{\mathsf{Enc}}, \underline{\boldsymbol{Y}}^{(b)}, \underline{\boldsymbol{\epsilon}}^{(b)}}, \underline{\boldsymbol{Y}}^{(b)}\big) + \mathsf{Reg}\big(\hat{\underline{\boldsymbol{Z}}}^{(b)}_{\boldsymbol{\theta}_{\mathsf{Enc}}, \underline{\boldsymbol{Y}}^{(b)}}, \underline{\boldsymbol{\sigma}}_{\boldsymbol{\theta}_{\mathsf{Enc}}}(\underline{\boldsymbol{Y}}^{(b)})\big),$$

which consists of

- ℓ_1 reconstruction error on raw data space with latent space perturbation
- Perceptual reconstruction error on image recognition NN space
- Terms penalizing too large latent representations and too small perturbation.

In the latter half, we add the following adversarial loss:

$$-\lambda \mathbf{1}^{\mathsf{T}} \mathsf{Discriminator}_{\boldsymbol{\theta}_{\mathsf{Disc}}}(\hat{\underline{\boldsymbol{Y}}}^{(b)}_{\boldsymbol{\theta}_{\mathsf{Dec}}, \boldsymbol{\theta}_{\mathsf{Enc}}, \underline{\boldsymbol{Y}}^{(b)}, \underline{\boldsymbol{\epsilon}}^{(b)}}).$$

Simultaneously, we minimize the following w.r.t. $\boldsymbol{\theta}_{\mathsf{Disc}}$:

$$\left\| [\mathbf{1} - \mathsf{Discriminator}_{\boldsymbol{\theta}_{\mathsf{Disc}}}(\underline{\boldsymbol{Y}}^{(b)})]_+ + [\mathbf{1} + \mathsf{Discriminator}_{\boldsymbol{\theta}_{\mathsf{Disc}}}(\hat{\underline{\boldsymbol{Y}}}^{(b)}_{\boldsymbol{\theta}_{\mathsf{Dec}}, \boldsymbol{\theta}_{\mathsf{Enc}}, \underline{\boldsymbol{Y}}^{(b)}, \underline{\boldsymbol{\epsilon}}^{(b)}})]_+ \right\|_1.$$

Question: How do we learn VAE in the image generation pipeline? Is it the same as the original VAE?

Answer:

- The AE idea makes the Decoder cover natural images.
- The VAE idea makes the Decoder be robust to perturbation in the latent space.
- The LPIPS idea makes the Decoder perceptually robust to perturbation.
- The adversarial learning idea makes the Decoder generate higher-quality images.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. When does the latent space strategy work?

4. How do we adapt various resolutions? - fully convolutional VAE

5. How does a VAE learn? How does it connect the latent space and raw data space?

6. Summary and Next Lecture

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. When does the latent space strategy work?

4. How do we adapt various resolutions? - fully convolutional VAE

5. How does a VAE learn? How does it connect the latent space and raw data space?

6. Summary and Next Lecture

Summary

- A map from low-dimensional $\mathcal{Z}$ to high-dimensional $\mathcal{I}$, whose image is a **subset** of the natural image domain. A decoder enables significant cost reduction since we only need to **generate the low-dimensional latent**.
- Fully-convolutional structure gives arbitrary output sizes by input sizing; as an autoencoder it has an approximate inverse; variational learning adds origin-concentration and robustness.
- The VAE in image generation pipeline is more than a vanilla VAE. It uses LPIPS and adversarial learning.

Next lecture

Next time, we will detail the **iterative generation**, which generates the appropriate low-resolution latent data.

References i

[1] Patrick Esser, Robin Rombach, and Bjorn Ommer.
Taming transformers for high-resolution image synthesis.
In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pages 12873–12883, 2021.

[2] Kenneth J. Falconer.
Fractal Geometry: Mathematical Foundations and Applications.
John Wiley & Sons, Chichester, 3 edition, 2014.

References ii

[3] Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio.
Generative adversarial nets.
In Advances in Neural Information Processing Systems (NeurIPS), pages 2672–2680, 2014.

[4] Diederik P. Kingma and Max Welling.
Auto-encoding variational bayes.
arXiv preprint arXiv:1312.6114, 2013.

References iii

[5] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer.
High-resolution image synthesis with latent diffusion models.
In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 10684–10695, 2022.

[6] Karen Simonyan and Andrew Zisserman.
Very deep convolutional networks for large-scale image recognition.
arXiv preprint arXiv:1409.1556, 2014.

References iv

[7] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. **The unreasonable effectiveness of deep features as a perceptual metric.** In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 586–595, 2018.